

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

Gustavo Duarte Ventino

**Concepção de um *Framework* para Geração de
Datasets
em Redes Domésticas para Sistemas IDS**

Uberlândia, Brasil

2026

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

Gustavo Duarte Ventino

**Concepção de um *Framework* para Geração de *Datasets*
em Redes Domésticas para Sistemas IDS**

Trabalho de conclusão de curso apresentado
à Faculdade de Computação da Universidade
Federal de Uberlândia, como parte dos requi-
sitos exigidos para a obtenção título de Ba-
charel em Ciência da Computação.

Orientador: Luís Fernando Faina

Universidade Federal de Uberlândia – UFU

Faculdade de Computação

Bacharelado em Ciência da Computação

Uberlândia, Brasil

2026

Gustavo Duarte Ventino

Concepção de um *Framework* para Geração de *Datasets* em Redes Domésticas para Sistemas IDS

Trabalho de conclusão de curso apresentado
à Faculdade de Computação da Universidade
Federal de Uberlândia, como parte dos requi-
sitos exigidos para a obtenção título de Ba-
charel em Ciência da Computação.

Trabalho aprovado. Uberlândia, Brasil, 06 de fevereiro de 2026:

Luís Fernando Faina
Orientador

Rafael Pasquini
Convidado 1

Bruno Augusto Nassif Travençolo
Convidado 2

Uberlândia, Brasil
2026

Resumo

O crescimento das redes domésticas, impulsionado pela proliferação de dispositivos IoT (*Internet of Things*) e pela digitalização do cotidiano, elevou significativamente a superfície de ataque disponível para agentes maliciosos. Paralelamente, a pesquisa em Sistemas de Detecção de Intrusão (IDS) baseados em aprendizado de máquina enfrenta a carência de *datasets* atualizados, rotulados e representativos do tráfego real de redes residenciais. Neste contexto, o presente trabalho tem como objetivo desenvolver uma arquitetura para geração de *datasets* voltados à detecção de anomalias e invasões em redes domésticas. A metodologia proposta fundamenta-se na configuração de um roteador com *firmware OpenWrt*, habilitando a funcionalidade de espelhamento de porta (*port mirroring*) para redirecionar o tráfego de rede a um servidor *Raspberry Pi*, responsável pela captura e processamento dos pacotes. A extração de características dos fluxos de rede é realizada pelo *CICFlowmeter*, enquanto a simulação de ataques, incluindo *Slow Loris* e mapeamento de rede, é automatizada por meio de contêineres *Docker* com endereçamento IP fixo, garantindo a rotulação determinística do *ground truth*. Os experimentos conduzidos em ambiente simulado e real, com ciclos de 12 e 24 horas, validam a estabilidade e a eficácia da solução. Os resultados demonstram a capacidade do sistema em operar de forma contínua sem degradação de desempenho, capturando a diversidade de protocolos característicos das redes domésticas modernas, incluindo tráfego de dispositivos móveis e IoT. A arquitetura desenvolvida apresenta-se como um *framework* completo para a criação de *datasets* customizáveis, contribuindo para o avanço das pesquisas em cibersegurança ao superar a obsolescência de bases de dados estáticas.

Palavras-chave: Redes domésticas, Detecção de intrusão, Detecção de anomalia, *Dataset*, Segurança cibernética.

Lista de ilustrações

Figura 1 – Explicação Visual de <i>port-mirroring</i>	14
Figura 2 – Topologia genérica proposta para a rede doméstica.	24
Figura 3 – Topologia proposta para a rede doméstica detalhada.	25
Figura 4 – Interfaces de rede disponíveis no roteador Archer C6 v3.	26
Figura 5 – Configuração padrão port-mirroring <i>port-mirroring</i>	27
Figura 6 – Configuração modificada port-mirroring <i>port-mirroring</i>	27
Figura 7 – Conteúdo de <i>argus_sample.csv</i>	33
Figura 8 – Comparativo entre Contêineres <i>Docker</i> e Máquinas Virtuais	36
Figura 9 – Rede doméstica com contêineres	38
Figura 10 – Visualização do algoritmo do contêiner <i>echo</i>	40
Figura 11 – Visualização do algoritmo do contêiner <i>scraper</i>	40
Figura 12 – Visualização do <i>script</i> de automação do ataque de mapeamento de rede.	43
Figura 13 – Visualização do <i>script</i> de automação do ataque <i>Slow Loris</i>	46
Figura 14 – Distribuição de Classes (Normal vs Anômalo)	53
Figura 15 – Série Temporal do Tráfego de Rede	53
Figura 16 – Top 10 Portas de Destino	54
Figura 17 – Matriz de Correlação entre Features	55
Figura 18 – Distribuição de Classes (24h)	57
Figura 19 – Série Temporal do Tráfego de Rede (24h)	57
Figura 20 – Top 10 Portas de Destino (24h)	59
Figura 21 – Matriz de Correlação entre Features (24h)	60
Figura 22 – Distribuição de Classes em Ambiente Real (12h)	62
Figura 23 – Série Temporal do Tráfego Real (12h)	62
Figura 24 – Top 10 Portas de Destino em Ambiente Real (12h)	63
Figura 25 – Matriz de Correlação em Ambiente Real (12h)	64
Figura 26 – Distribuição de Classes em Ambiente Real (24h)	66
Figura 27 – Série Temporal do Tráfego Real (24h)	67
Figura 28 – Top 10 Portas de Destino em Ambiente Real (24h)	68
Figura 29 – Matriz de Correlação em Ambiente Real	69

Lista de tabelas

Tabela 1 – Representação de <i>tshark_sample.csv</i> como tabela	32
Tabela 2 – Representação de <i>cicflowmeter_sample.csv</i> como tabela	34
Tabela 3 – Resultados do teste de geração de tráfego	41
Tabela 4 – Resultados do teste de geração de tráfego anômalo	50
Tabela 5 – Resumo geral das métricas do experimento com tráfego simulado de média duração	52
Tabela 6 – Top 10 portas de destino mais acessadas no experimento com tráfego simulado de média duração	54
Tabela 7 – Resumo geral das métricas do experimento com tráfego simulado de longa duração	56
Tabela 8 – Top 10 portas de destino mais acessadas no experimento com tráfego simulado de longa duração	58
Tabela 9 – Resumo geral das métricas do experimento em ambiente real de média duração	61
Tabela 10 – Top 10 portas de destino mais acessadas no experimento em ambiente real de média duração	63
Tabela 11 – Resumo geral das métricas do experimento em ambiente real de longa duração	65
Tabela 12 – Top 10 portas de destino mais acessadas no experimento em ambiente real de longa duração	67

Lista de abreviaturas e siglas

CLI	<i>Command Line Interface</i>
CSV	<i>Comma-Separated Values</i>
DNS	<i>Domain Name System</i>
DoS	<i>Denial of Service</i>
FTP	<i>File Transfer Protocol</i>
HTTP	<i>HyperText Transfer Protocol</i>
HTTPS	<i>HyperText Transfer Protocol Secure</i>
ICMP	<i>Internet Control Message Protocol</i>
IDS	<i>Intrusion Detection System</i>
IoT	<i>Internet of Things</i>
IP	<i>Internet Protocol</i>
ISP	<i>Internet Service Provider</i>
LAN	<i>Local Area Network</i>
LTS	<i>Long Term Support</i>
mDNS	<i>Multicast DNS</i>
ML	<i>Machine Learning</i>
NTP	<i>Network Time Protocol</i>
OEM	<i>Original Equipment Manufacturer</i>
PCA	<i>Principal Component Analysis</i>
PCAP	<i>Packet Capture</i>
QUIC	<i>Quick UDP Internet Connections</i>
RAM	<i>Random Access Memory</i>
SFTP	<i>Secure File Transfer Protocol</i>

SOHO	<i>Small Office/Home Office</i>
SSH	<i>Secure Shell</i>
SSDP	<i>Simple Service Discovery Protocol</i>
SVM	<i>Support Vector Machine</i>
TCP	<i>Transmission Control Protocol</i>
TLS	<i>Transport Layer Security</i>
UDP	<i>User Datagram Protocol</i>
UFW	<i>Uncomplicated Firewall</i>
VPN	<i>Virtual Private Network</i>
WAN	<i>Wide Area Network</i>

Sumário

1	INTRODUÇÃO	10
2	FUNDAMENTAÇÃO TEÓRICA	12
2.1	Avaliação do <i>Firmware</i> do Roteador Doméstico	12
2.1.1	O Roteador enquanto Elemento de Rede	13
2.2	Captura e Análise de Pacotes de Rede (<i>Sniffing</i>)	14
2.3	Formatação dos Dados para <i>Machine Learning</i>	15
3	TRABALHOS RELACIONADOS	17
3.1	Avaliação do <i>Firmware</i> do Roteador Doméstico	17
3.2	Captura e Análise de Pacotes de Rede (<i>sniffing</i>)	18
3.3	Formatação dos Dados para <i>Machine Learning</i>	19
4	MÉTODO	22
4.1	Avaliação do <i>Firmware</i> do Roteador	22
4.1.1	Substituindo <i>Firmware</i> do Roteador	23
4.1.2	Ajustes na Topologia da Rede	24
4.1.3	Configurando o Roteador	25
4.2	Configurando Servidor <i>Sniffer</i>	27
4.2.1	Escolha do Servidor	28
4.2.2	Configuração do SSH, FTP e Outros Serviços	28
4.2.3	Configurando <i>Sniffer</i>	29
4.3	Geração de Tráfego	35
4.3.1	Geração de Tráfego Normal	35
4.3.2	Geração de Tráfego Anômalo	41
4.3.2.1	Mapeamento de Rede	42
4.3.2.2	<i>Denial of Service – Slow Loris</i>	45
4.3.2.3	Ambiente de Execução e Identificação dos Ataques	47
4.3.2.4	Validação Preliminar da Geração de Tráfego Anômalo	49
4.4	Experimentos	50
4.4.1	Experimentos com Tráfego Simulado	51
4.4.1.1	Experimento de Média Duração (12 Horas)	52
4.4.1.2	Experimento de Longa Duração (24 Horas)	56
4.4.2	Experimentos em Ambiente Real	60
4.4.2.1	Experimento de Média Duração (12 Horas)	61
4.4.2.2	Experimento de Longa Duração (24 Horas)	64

5	CONCLUSÃO	70
5.1	Trabalhos Futuros	71
	REFERÊNCIAS	72

1 Introdução

As redes domésticas, que antes eram utilizadas para atividades como navegação casual na Internet e *streaming* de mídia, agora são a base das vidas digitais de seus usuários. Isto ocorre pois essas redes suportam e garantem a coesão entre uma variedade de dispositivos, desde computadores e *smartphones* até instrumentos IoT (*Internet of Things*), como assistentes de voz, câmeras de segurança e eletrodomésticos inteligentes que apresentam facilidades no cotidiano de seus usuários (FERNANDES; JUNG; PRAKASH, 2016). Assim, paralelamente ao crescimento das redes domésticas em número de dispositivos e tráfego de dados, houve o aumento das ameaças encontradas nestes ambientes (DACIER et al., 2017).

Além disso, no Brasil, foram registradas mais de 88,5 bilhões de tentativas de ataques cibernéticos durante o ano de 2021 (LABS, 2022). Portanto, a necessidade de proteger essas redes contra ameaças torna-se cada vez mais evidente. Com o aumento da quantidade de dados pessoais que os aparelhos conectados às redes residenciais processam e armazenam, torna-se necessária a garantia de soluções que assegurem a segurança das redes domésticas e das informações que por elas trafegam. Com base nas pesquisas anteriormente citadas, é possível notar que há uma preocupação dos pesquisadores em relação à segurança em redes residenciais, e que a comunidade científica busca soluções para os problemas presentes neste contexto.

Nos últimos anos, uma ampla gama de trabalhos demonstrando a preocupação em propor soluções para garantir a segurança das redes domésticas e das informações que por elas trafegam vem sendo desenvolvida. Trabalhos como os de (ZAIDAN; ZAIDAN, 2018) e (MASCARENHAS et al., 2021) listam algumas preocupações, tais como a conectividade dos dispositivos, limitações de *hardware*, conduta do usuário e problemas de segurança. Ademais, é possível citar diversos trabalhos que propõem arquiteturas para redes residenciais, como o trabalho de pesquisa europeu “*Safe Guarding Home IoT Environments with Personalised Realtime Risk Control GHOST*” (COLLEN, 2018) e “*Secure Smart Home Environment (SHSec)*” (SHARMA et al., 2019). Esses trabalhos têm como objetivo principal propor soluções para problemas específicos, tais como: violações de segurança em dispositivos IoT, detecção de *malwares* e classificação de usuários e dispositivos.

Neste contexto, os objetivos gerais do trabalho são: estudar mecanismos de coleta dos pacotes que trafegam pela rede doméstica, entender o melhor formato de armazenar esses pacotes, descobrir quais são as informações relevantes a serem extraídas dos pacotes coletados e quais medidas estatísticas podem ser abstraídas destes. Além disso, há a necessidade de gerar tráfego normal e anômalo, para que ao final do trabalho um *dataset*

com informações suficientes para o treinamento de máquinas de aprendizado voltadas para a detecção de intrusões, ataques e anomalias em redes domésticas possa ser definido.

Considerando os objetivos principais do trabalho, algumas ferramentas computacionais que se relacionam diretamente com esses objetivos são necessárias. Para coletar todos os pacotes que trafegam pela rede e armazená-los, é preciso analisar o *firmware* do roteador doméstico para avaliar se ele permite o redirecionamento ou duplicação dos pacotes. Para isso, pode ser necessária a instalação e configuração de um *firmware* alternativo, como o *OpenWRT* (*Open Wireless Router*) ([OpenWrt Project, 2024](#)) ou o *DD-WRT* (*Dresden Wireless Router*) ([DD-WRT, 2024](#)), no roteador principal da rede doméstica, em conjunto com os respectivos pacotes de *port mirroring*.

Além disso, é imprescindível a presença de um servidor *Linux* para atuar como dispositivo de coleta e análise de pacotes, podendo este ser um *Raspberry Pi* ou outro computador de placa única com capacidade similar. É necessário também o uso de *softwares* específicos para captura do tráfego, como *Tshark* ([Tshark, 2024](#)) e *CICFlowmeter* ([Lashkari, Arash Habibi, 2016a](#)), além da configuração de ferramentas para acesso remoto ao servidor, como SSH (*Secure Shell*). Por fim, há necessidade do uso de ferramentas para simular ataques e intrusões de maneira automatizada, bem como ferramentas voltadas para gerar tráfego normal e anômalo.

Utilizando esses componentes de *hardware* e *software*, o objetivo é replicar, por meio do roteador principal da rede doméstica, os pacotes que trafegam pela porta WAN do roteador para a porta LAN na qual o servidor está conectado. Isso permite que a ferramenta de coleta capture e armazene os pacotes. Com os pacotes armazenados, são analisados quais campos são relevantes para a categorização por uma máquina de aprendizado, e são extraídas novas informações dos dados contidos nos pacotes, processo conhecido como formatação dos dados coletados.

2 Fundamentação Teórica

A monografia está organizada em seções. Esta seção trata da fundamentação teórica, na qual são apresentados os principais conceitos teóricos nos quais este trabalho se sustenta, assim como alguns detalhes da implementação que devem ser apresentados para melhor entendimento do desenvolvimento do trabalho. Assim, esta seção é dividida em: Seção 2.1 - Avaliação do *Firmware* do Roteador Doméstico, 2.2 - Captura e Análise de Pacotes de Rede (*Sniffing*), 2.3 - Formatação dos Dados para *Machine Learning*”.

2.1 Avaliação do *Firmware* do Roteador Doméstico

Nesta seção, é abordada a importância do roteador como elemento fundamental na topologia da rede doméstica. Além disso, as características e especificações do roteador doméstico utilizado no trabalho são descritas nesta seção, considerando sempre o papel fundamental que este elemento de rede desempenha no desenvolvimento deste trabalho, também é comentada a importância de entender e modificar a topologia da rede doméstica para permitir a captura de pacotes.

Os roteadores são os elementos responsáveis por conectar duas ou mais redes de computadores que se comunicam por meio de pacotes. Isto é, tudo que entra ou sai de uma rede local, ou subrede, deve passar por um roteador, o qual provê a rota para que haja a comutação de pacotes. Fica evidente, portanto, a importância dos roteadores na captura dos dados que fluem pelas redes domésticas, uma vez que todos pacotes que entram na rede passam por tais elementos, assim como todos os pacotes que saem da rede.

Uma vez que são responsáveis por direcionar e orientar os pacotes da rede, entende-se que os roteadores são dispositivos essenciais nas redes domésticas. Este manejo de pacotes é feito pelo *firmware* do roteador, sendo assim, é necessário entender o que é um *firmware*. Comumente, o conceito de *firmware* é confundido com *software*. Entretanto, o *firmware* é na verdade uma classe de *Software*, que provê instruções para que o *Hardware* seja capaz de inicializar corretamente, se comunicar com outros dispositivos e realizar atividades básicas, como lidar com entrada e saída de dados. Em suma, trata-se de um programa embarcado no *hardware* que permite operá-lo efetivamente ao controlar e gerir os elementos físicos. Tendo isto em vista, entende-se que roteadores possuem *firmware*, ou *software* embarcado, nativo que possibilita o funcionamento apropriado do *hardware*.

2.1.1 O Roteador enquanto Elemento de Rede

Antes de partir para a Seção 2.2, é necessário estabelecer alguns conceitos fundamentais que permitem que a ideia de capturar todo o tráfego de entrada e saída do roteador doméstico amadureça. Os roteadores são elementos de terceira camada de rede, isto é, são responsáveis por determinar como os pacotes são roteados da origem até o destino. Para isso, os roteadores devem ser constituídos de um *hardware* que possibilite que isso seja feito ou da melhor forma possível, ou da forma mais aceitável para o usuário. Além desse papel fundamental de traçar a rota que os pacotes farão pela rede, os roteadores também são utilizados como pontos de acesso a redes sem fio, disponibilizando largura de banda para dispositivos *Wi-Fi*.

Dito isso, os roteadores domésticos comumente disponibilizam dois enlaces diferentes: um enlace relacionado diretamente à rede cabeada, associado às portas WAN (*Wide Area Network*) e às portas LAN (*Local Area Network*), e um enlace associado às antenas *Wi-Fi*. O enlace da rede cabeada normalmente é utilizado para conectar dispositivos que só podem se conectar e comunicar por meio da rede cabeada, como *switches*, *hubs* e outros roteadores, ou para conectar dispositivos que desfrutam da alta velocidade dos dados e da baixa propensão a ruídos da rede cabeada, tal qual servidores.

O enlace relacionado à rede sem fio é utilizado para conectar dispositivos que possuem módulos *Wi-Fi* e são capazes de se conectar a essas redes, como celulares, dispositivos IoT (*Internet of Things*), computadores pessoais e afins. Sendo os roteadores compostos comumente por esses dois enlaces segmentados, a ideia de capturar tráfego seria limitada a um desses enlaces. Isto é, em tese ou se captura somente tráfego do enlace cabeado, ou somente do enlace *wireless*. Entretanto, alguns roteadores possuem um terceiro enlace, comumente chamado de *br-lan* ou *bridge*, o qual atua como uma ponte entre os enlaces cabeado e *wireless*, tal qual o nome sugere. Isto é, este enlace é o responsável por permitir que pacotes sejam roteados entre dispositivos conectados ao enlace cabeado com dispositivos conectados ao enlace *wireless*, funcionando tal qual um atalho que facilita a comunicação entre dispositivos conectados no mesmo roteador, mas em enlaces diferentes.

Mas como isto é feito? Este enlace comumente é implementado como um barramento unificado controlado pelo *firmware* do roteador. De forma que todas as portas LAN disponíveis no roteador, incluindo a porta WAN, se conectem com as antenas *Wi-Fi* pelo barramento controlado pelo *firmware*. Dessa forma, o tráfego que passa por qualquer um destes enlaces obrigatoriamente passa por este barramento. Assim, a ideia de capturar todo o tráfego que passa pelo roteador a nível de pacote ganha maturidade, uma vez que basta capturar o tráfego deste enlace *bridge*, caso o roteador disponha de tal enlace, para ter acesso a todo o tráfego de entrada e saída do roteador.

Para finalizar, é necessário compreender que capturar este tráfego diretamente por meio deste barramento *bridge* é inviável. Isso se dá pois *sniffers* como *tshark* (Tshark, 2024) dispõem somente de captura via interfaces de rede às quais o servidor *sniffer* pode se conectar, sendo estas as interfaces físicas (LAN) e *wireless* (WLAN). Entretanto, é possível espelhar todo o tráfego da interface *bridge* do roteador para outra por meio de uma ferramenta chamada *port-mirroring*, descrita na Figura 1, entretanto nem todos os roteadores dispõem de tal funcionalidade nativamente. Caso o roteador escolhido não disponha de tal funcionalidade nativamente, faz-se necessário investigar a compatibilidade deste roteador com *firmwares* alternativos que suportem esta funcionalidade, como *OpenWRT* (OpenWrt Project, 2024), para que seja possível a captura do tráfego da rede doméstica.

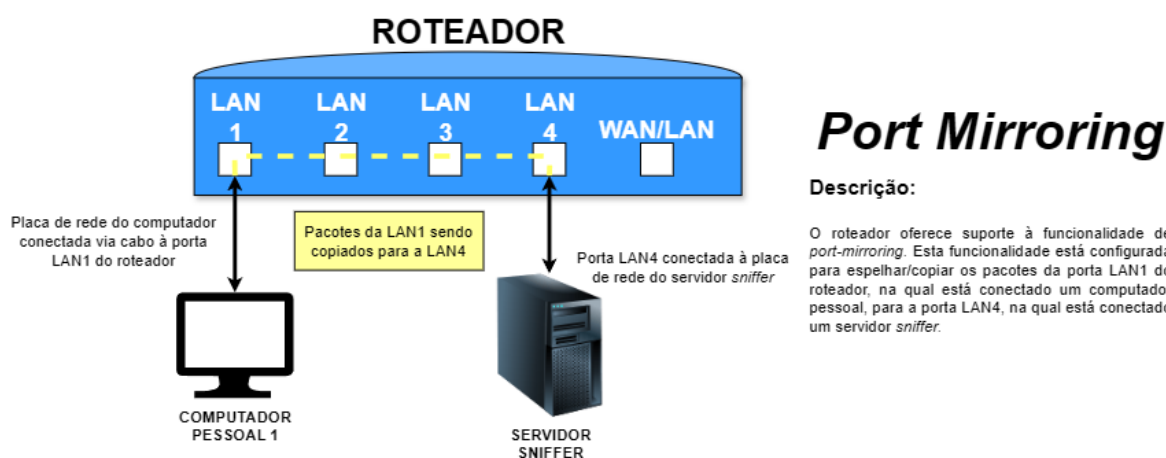


Figura 1 – Explicação Visual: *port-mirroring*. Fonte: Do Autor.

2.2 Captura e Análise de Pacotes de Rede (*Sniffing*)

Nesta seção, é abordada a definição de *Packet Sniffing* e sua importância no contexto de pesquisa em cibersegurança. Também é apresentado um breve histórico dos *Sniffers*, a fim de contextualizar o leitor e apresentar as ferramentas utilizadas no desenvolvimento deste trabalho. Além disso, também é abordada toda a estrutura dos pacotes de rede, explicando e comentando camada a camada dos pacotes, assim como os diversos formatos de arquivo disponíveis para registrar uma captura de pacotes e os ajustes necessários na topologia da rede para que seja possível "farejar" pacotes com excelência.

Packet Sniffing, também conhecido como *Sniffing*, é uma técnica que permite a captura e análise de pacotes que circulam em uma rede. Normalmente, a captura é realizada por meio de *switches*. No entanto, na ausência destes, é viável configurar um recurso conhecido como espelhamento de portas no roteador (*port mirroring*). Este recurso possibilita que todos os pacotes sejam duplicados de uma porta específica do roteador para outra. A partir disto, é possível o uso de uma ferramenta comumente usada para a análise

e captura de pacotes, o *Wireshark* ([WIRESHARK, 2024](#)), que faz uso de qualquer placa de rede registrada no *site* oficial para seu funcionamento.

Nos anos 80 os *sniffers* precisavam de módulos dedicados de *hardware* como o *Nestar ARCNET Sniffer* ([NESTAR, 1984](#)). Tal *hardware* não é mais necessário, programas como o *Wireshark* ([WIRESHARK, 2024](#)) e sua versão CLI (*Command Line Interface*) *Tshark* ([Tshark, 2024](#)) executam em diversos computadores sem precisar de uma placa de rede dedicada. Ademais, a importância da Internet aumentou muito, ela é empregada para uma variedade de atividades: transferências monetárias, compras, troca de mensagens de texto e voz, compartilhamento de imagens, entre outros. Tal fato evidencia a necessidade de tornar as redes domésticas mais seguras, objetivo para o qual *sniffing* é útil.

Isso se dá pois a técnica de análise de pacotes é empregada para diversos propósitos em várias áreas, por exemplo, gerentes de rede podem utilizá-la com o objetivo de monitorar e documentar o tráfego e detectar atividade suspeita. Até mesmo criminosos podem se valer dessa técnica para buscar informações sensíveis sem autorização, assim como cientistas e pesquisadores podem fazer uso dos pacotes coletados para treinar máquinas de aprendizado e construir *datasets*, a fim de atingir objetivos específicos, como a criação de um sistema que detecta anomalias.

Uma vez que os pacotes são capturados, há uma série de dados nativamente encapsulados dentro destes pacotes, como: indicações dos protocolos utilizados, campos de cabeçalho, mensagens, *timestamps*, números de sequência, dentre outros. A questão é que esses dados devem ser desdobrados e formatados, pois caso estes dados cheguem ao *dataset* final sem refinamento adequado, impactam a utilidade e usabilidade deste. Esta etapa é mais detalhada na Seção 2.3, na qual também é abordado quais características devem ser abstraídas dos dados nativos dos pacotes para a construção de um *dataset* eficiente para detecção de anomalias.

2.3 Formatação dos Dados para *Machine Learning*

Nesta seção, é abordada a importância de uma formatação apropriada dos dados ao construir um *dataset*. Além disso, são apresentadas as principais dificuldades em construir um *dataset* a partir do fluxo de pacotes de rede, e superficialmente como estas são contornadas no desenvolvimento. Nesta seção também é apresentada a definição de dado normal e dado anômalo, assim como a importância destes dados num *dataset* voltado para redes e cibersegurança.

Para construir um *dataset* para detecção de anomalias em redes domésticas, que possibilite a análise e classificação do tráfego de rede por meio dos pacotes coletados, é imprescindível formatar os dados de maneira apropriada. Conforme destacado em *Pré-processamento de Dados em Aprendizado de Máquina Supervisionado* ([BATISTA, 2003](#)),

a qualidade dos dados é uma preocupação primordial no Aprendizado de Máquina, sendo crucial saber como manejar situações em que as informações necessárias estão incompletas ou ausentes. Por isso, faz-se necessário formatar os dados nativos dos pacotes.

Os dados coletados devem ser formatados para o formato que se adeque a máquinas de aprendizado, que comumente seguem padrão de dados abertos, como em arquivos CSV (*Comma-separated Values*), nos quais cada linha representa uma amostra e cada coluna, uma característica da amostra. Portanto, é necessário compreender, por meio dos pacotes coletados, quais dados são relevantes para compor as colunas diretamente e quais são pertinentes apenas se abstraídos, ou seja, utilizados para a criação de novas colunas após um processo de abstração, como um cálculo específico que resulta em um indicador valioso.

Ao lidar com os dados em nível de pacote, tem-se um volume de pacotes muito alto, o que gera um *dataset* com muitas amostras. Sendo assim, entende-se que a criação de campos de valores agregados como média, desvio padrão e variância é necessária para agrupar alguns pacotes, a fim de reduzir parcialmente a quantidade de amostras. Além disso, cada pacote possui muitos campos, o que gera um *dataset* com alta dimensionalidade. Portanto, entende-se que é de suma importância para a definição de um *dataset* apropriado, uma etapa de *Feature Selection* e a aplicação de algum algoritmo de redução de dimensionalidade como o PCA (*Principal Component Analysis*) (MAĆKIEWICZ; RATAJCZAK, 1993), a fim de reduzir a dimensionalidade do *dataset*.

3 Trabalhos Relacionados

3.1 Avaliação do *Firmware* do Roteador Doméstico

Improving Network Security Using DD-WRT as a Solution for SOHO Routers (MARIAN; DORU, 2023) deixa evidente a necessidade de avaliação do *firmware* do roteador doméstico. Isto é, é analisado se o *software* embarcado nos roteadores SOHO (*Small Office/Home Office*) abrange todas as funcionalidades requeridas para a construção de uma rede segura. Para isso, avalia-se quais são as funcionalidades disponibilizadas pelo fabricante do roteador. Funcionalidades como: VPN (*Virtual Private Network*), configurações de *firewall*, e espelhamento de portas, permitem a melhora da segurança da rede se bem implementados e devidamente configurados.

A pesquisa avalia o *firmware* nativo do roteador TP-Link AC1750, modelo Archer C7, evidenciando a ausência das ferramentas necessárias para aumentar a segurança da rede. Neste caso, como tais ferramentas não estavam disponíveis no *firmware* do fabricante, é proposta a substituição deste *firmware* por outro. Assim, alguns firmwares devem ser avaliados a fim de encontrar algum que contemple as funcionalidades requeridas para elevar os níveis de segurança da rede. Neste caso, algumas opções são mais comuns, como o DD-WRT (DD-WRT, 2024) e o OpenWrt (OpenWrt Project, 2024), os quais disponibilizam as funcionalidades mencionadas. Para que o *firmware* possa ser escolhido, é necessário avaliar qual oferece o devido suporte ao *hardware* disponível, e qual melhor atende aos requisitos do trabalho.

Feita a escolha do *firmware*, é ressaltada a importância de fazer uma análise completa da rede em seu estado atual. Isto deve ser feito pois é necessário configurar o *firmware* de maneira adequada, minimizando conflitos com outros dispositivos e evitando comprometer a estrutura da rede. Uma vez que a estrutura atual da rede foi analisada, é possível fazer a atualização do *firmware* do roteador.

Uma vez que o *firmware* foi substituído, deve-se fazer a configuração com base na documentação oficial e na análise anterior da rede. Após a configuração inicial, deve-se configurar as funcionalidades às quais motivaram a avaliação dos *firmwares* disponíveis. Com tudo devidamente configurado, chegou o momento de testar possíveis vulnerabilidades na rede, por meio do escaneamento de portas por exemplo. Além disso, a estabilidade da rede, e outros fatores como velocidade, devem ser testados, a fim de garantir que a atualização foi bem-sucedida. Dessa forma, é possível prevenir ataques cibernéticos, buscas por vulnerabilidades, e o desenvolvimento de ferramentas capazes de detectar intrusões.

Este trabalho é relevante para a presente pesquisa uma vez que aborda todo o

processo de avaliação dos possíveis *firmwares* e como o *firmware* do roteador desempenha um papel fundamental quanto à segurança da rede. Além disso, o trabalho faz também todo o processo de substituição do *firmware* de fábrica pelo DD-WRT, e comenta todos os passos envolvidos no processo, como análise prévia da rede, configuração inicial, e configuração das funcionalidades requeridas para aumentar os níveis de segurança.

3.2 Captura e Análise de Pacotes de Rede (*sniffing*)

Analysis for Network Forensics: A comprehensive Survey (SIKOS, 2020), aborda como a coleta e análise de pacotes (*Packet Sniffing*), incluindo métodos de inspeção detalhada (*Deep Packet Inspection*), são de extrema relevância na análise forense de redes. O texto se inicia, apresentando diferentes formas de capturar o fluxo de pacotes de uma rede, partindo de duas premissas. A primeira situação descrita é a de redes que usam *Hubs*, nestas a instalação de um *sniffer* torna-se banal em detrimento do funcionamento padrão dos *Hubs*. A segunda situação descrita é a de redes que usam *switches*, nestas existem muitas maneiras possíveis de capturar o fluxo de pacotes, entretanto, a mais consistente e apropriada para maior parte dos casos é *port mirroring*. O Espelhamento de Portas, ou *port mirroring*, é indicado pois muitos *switches* de rede oferecem esta funcionalidade por padrão, assim como muitos roteadores também oferecem tal ferramenta para os administradores da rede. Além disso outros métodos como: *Network Tapping*, e *Hubbing Out*, são apresentados como alternativas, mas apresentam pontos negativos que os tornam inviáveis dependendo do cenário.

Network Tapping, consiste no uso de conectores físicos do tipo vampiro para capturar o tráfego direcionado a uma das portas. No cenário doméstico, no qual se pretende capturar todo o tráfego do roteador principal da rede, esse método é inviável. Isto ocorre pois não há possibilidade de capturar o tráfego do cabo de rede par trançado com um conector vampiro físico, ou seja, é impossível capturar tudo que entra e sai da rede. *Hubbing Out* consiste em centralizar todo o tráfego em um *Hub*, ao qual o conector vampiro lógico deve estar conectado e capturando os pacotes. Tal método é inviável neste cenário, pois não é possível substituir o roteador principal da rede por um *Hub*. O *Port Mirroring* é capaz de capturar todo o tráfego da rede no cenário descrito, pois é configurado para espelhar o tráfego da porta WAN (*Wide Area Network*) para a porta LAN (*Local Area Network*) à qual o *sniffer* está conectado.

Isto é, uma vez que os pacotes foram capturados por aplicações de *Sniffing* como o *Wireshark* (WIRESHARK, 2024) e *Tcpdump* (The Tcpdump Group, 2025), é necessário armazená-los num formato apropriado, comumente *Packet Capture* (PCAP). Uma vez que os pacotes capturados são armazenados, é possível analisar as informações contidas, e avaliar quais são relevantes para um modelo de aprendizado de máquina. Nesta etapa

de formatação dos dados, os pacotes capturados e armazenados são transformados em linhas de uma tabela, usualmente no formato *Comma Separated Values* (CSV). Enfim, para que os pacotes sejam categorizados, é necessário que esta tabela passe por um pré-processamento, o qual permite o seu uso no treinamento de máquinas de aprendizado.

Este trabalho relaciona-se com a presente pesquisa em razão de abordar diferentes formas de captura de pacotes, indicar as mais apropriadas para o cenário doméstico, enquanto ressalta a importância da análise de pacotes para o monitoramento e detecção de intrusões em redes. Além disso, são apresentadas algumas noções básicas de inspeção dos pacotes e formatação dos dados, uma vez que as máquinas de aprendizado necessitam dos pacotes coletados num formato específico para a construção do conhecimento necessário para classificação do tráfego interno às redes.

Além disso, existem outras ferramentas de captura de tráfego de rede, como o *Argus* (OPENARGUS, 2024). Essa ferramenta captura o tráfego de forma diferente das ferramentas habituais, como o *Wireshark* (WIRESHARK, 2024) e o *tcpdump* (The Tcpdump Group, 2025). Em vez de apresentar as informações desencapsuladas dos pacotes, ela processa esses dados e apresenta um resumo do fluxo de dados. Em suma, trata-se de uma boa ferramenta para analisar aspectos dos dados que não podem ser facilmente obtidos com analisadores de pacotes mais convencionais, podendo ser utilizada como um recurso auxiliar no fluxo de captura e análise de pacotes. Além disso, o *Argus* conta com ferramentas de geração de pacotes, o que é extremamente útil, uma vez que há necessidade de gerar tráfego para construção do *dataset* que está sendo proposto neste trabalho.

3.3 Formatação dos Dados para *Machine Learning*

A *Survey of Feature Selection and Feature Extraction Techniques in Machine Learning* (KHALID; KHALIL; NASREEN, 2014), descreve que conjuntos de dados com muitas dimensões, normalmente possuem muitos atributos irrelevantes, ou redundantes, que aumentam o espaço de busca, e dificultam o processamento de dados. Isto dificulta o processo de descoberta em *Knowledge Discovery in Databases* (KDD), pois há pouquíssimos dados relevantes que contribuem com a construção de conhecimento, e muitos dados irrelevantes para esta etapa, a qual é fundamental. Sendo assim, existem uma série de algoritmos de seleção e extração de atributos, para melhorar o processo de aprendizado.

As técnicas para a seleção de atributos são responsáveis por filtrar ou condensar atributos. Existem inúmeros algoritmos capazes de seleção de atributos. Esta classe de algoritmos é chamada de *Feature Selection Algorithms* (FSA), na qual há algoritmos gerados a partir de uma definição de relevância, responsáveis pela seleção de atributos. Estes algoritmos podem ser divididos em três categorias: Filtros, *Wrappers*, e Híbridos.

Os algoritmos de filtragem possuem custo computacional baixo, são rápidos, e não

oferecem confiança eficiente na classificação, portanto são indicados para conjuntos de dados com muitas dimensões. Os algoritmos *wrappers* oferecem um desempenho melhor que os algoritmos de filtragem, pois o processo de seleção de atributos é otimizado. Entretanto, estes métodos têm alto custo computacional para serem usados em conjuntos de muitas dimensões, isso se dá pois cada classificador, que é responsável por otimizar a seleção, deve avaliar cada atributo disponível. Os algoritmos híbridos por sua vez, usam uma combinação de características dos algoritmos *wrappers* e dos algoritmos de filtragem, numa tentativa de balancear desempenho e custo computacional.

Além disso, há extração de atributos, que é descrita como o processo de obtenção de atributos capazes de refletir o conteúdo intrínseco de um conjunto de dados coletados. Enquanto isso, a seleção de atributos é descrita como o processo de seleção de atributos relevantes, isto é, remoção de atributos irrelevantes e de dados redundantes. Entretanto, para que seja possível definir quais são os atributos usados nestas etapas, é necessário que os dados estejam em um formato de dados abertos, como o *Comma Separated Values* (CSV), no qual são construídas tabelas nas quais cada linha corresponde a um conjunto de pacotes coletados, e cada coluna a um atributo deste conjunto.

Para a extração de atributos, é necessária a aplicação de algumas técnicas de extração de atributos, como a *Transformation*. Esta técnica consiste em transformar atributos originais, a fim de gerar novos atributos que são mais significativos. A solução mais popular para a extração de atributos é a *Principle Component Analysis* (PCA), que é usado para extrair as informações mais relevantes de um conjunto de dados redundantes, ou de dados com ruído. Este método consiste em uma transformação linear de dados que minimiza a redundância, à qual é medida pela covariância, e maximiza as informações relevantes, medidas pela variância.

Uma das maneiras de fazer a formatação de dados de maneira apropriada, isto é, decidir quais algoritmos devem ser aplicados, e qual é o resultado que se deseja alcançar, consiste em basear-se em conjuntos os quais já passaram por análise e validação científica, como o: KDD-99 ([ARCHIVE, 1999](#)), o UNSW-NB15 ([MOUSTAFA, N.; SLAY, J., 2015](#)), e o CIC-IDS2017 ([CYBERSECURITY, 2017](#)). O trabalho *The Significant Features of the UNSW-NB15 and the KDD99 Data Sets for Network Intrusion Detection Systems* ([MOUSTAFA; SLAY, 2015](#)) aborda quais informações são mais relevantes nos *datasets* KDD-99 e UNSW-NB15, e pode ser usado como uma referência no momento de construção do *dataset* que está sendo proposto neste trabalho. Por exemplo, diante da análise de referências como ([MOUSTAFA; SLAY, 2015](#)) torna-se evidente que algumas informações devem ser extraídas, como a média, o desvio padrão, o valor mínimo, e o valor máximo de alguns campos específicos dos pacotes, a fim de realizar um bom agrupamento destes.

Abordar estes tópicos é de suma importância pois a maioria dos algoritmos de aprendizado supervisionado adquirem conhecimento restritamente a partir dos dados co-

letados. Isto é, a qualidade do conhecimento adquirido é determinada pela qualidade dos dados de entrada, assim como a qualidade dos dados de entrada é determinada pela qualidade dos dados coletados, de sua formatação, e de seu pré-processamento.

4 Método

Neste capítulo é abordado o método sob o qual o trabalho foi desenvolvido, dando ênfase nos processos realizados para que os resultados desejados fossem obtidos, isto é, aborda-se tudo desde a avaliação do *firmware* do roteador até os ajustes realizados na topologia da rede e configuração do roteador a fim de possibilitar a captura de pacotes. Vale ressaltar que todos os códigos e *scripts* apresentados durante este trabalho estão disponíveis no repositório público "*ProjectBeholder*" (VENTINO, 2026).

4.1 Avaliação do *Firmware* do Roteador

Existem casos em que o *firmware* não contemple todos os requisitos funcionais necessários para a replicação de pacotes, como a falta de funcionalidades específicas, e restrições que impedem o desenvolvimento de trabalhos e pesquisas, portanto faz-se necessária uma Avaliação do *firmware* do roteador. Esta etapa de avaliação consiste em uma pesquisa breve acerca do roteador disponível para uso na rede doméstica. Buscas envolvendo o modelo do roteador e funcionalidades que são tidas como pré-requisitos, tal qual o suporte à *port-mirroring*, costumam ser mais do que suficientes. Dessa forma, torna-se possível descobrir informações técnicas das capacidades do *firmware* padrão do roteador.

Os resultados dessa pesquisa determinam se o roteador necessita ou não de um *firmware* alternativo para que o trabalho tenha continuidade. Caso o *firmware* corrente não contemple todas as funcionalidades necessárias, comumente é possível usar configurações de fábrica para recuperação do *firmware* nativo ou métodos paralelos, para a instalação de *firmwares* alternativos, como *OpenWRT* (OpenWrt Project, 2024) e *DD-WRT* (DD-WRT, 2024), que podem suprir as necessidades do usuário. Este processo é chamado de Atualização de *firmware* do Roteador, entretanto nem todos os roteadores possuem *hardware* e *firmware* nativo compatíveis com este processo, o que pode levar à aquisição de um roteador específico para a construção de *datasets* em redes domésticas.

Durante o desenvolvimento do trabalho, o *modem*/roteador inicialmente disponível pelo ISP: "SAGEMCOM F@ST 3896", não oferecia a funcionalidade de *port-mirroring*, tal qual não suportava o processo de atualização do *firmware* nativo por um alternativo. Dessa forma foi necessário adquirir um outro roteador que contemplasse estes requisitos para o desenvolvimento efetivo do trabalho, este processo de aquisição é descrito detalhadamente na Seção 4.1.1.

4.1.1 Substituindo *Firmware* do Roteador

Para realizar a substituição do *firmware* nativo por um paralelo, as especificações de *hardware* do roteador devem ser avaliadas, a fim de descobrir se este é compatível com as versões mais recentes do *firmware* escolhido. Para isso, faz-se necessário verificar o *website* fornecedor do *firmware* a fim de verificar a tabela de *hardware* suportado oficialmente. No desenvolvimento deste trabalho optou-se pela substituição do *firmware* nativo pelo *OpenWRT*, esta decisão ocorreu em razão de alguns motivos, estes são: grande variedade de dispositivos suportados, alta qualidade da documentação oficial, suporte nativo ao pacote "*port-mirroring*" (MARAYA, 2024) e fácil interação com os desenvolvedores via fórum.

Dito isso, o primeiro passo para escolha do roteador foi investigar a lista de modelos suportados oficialmente. Este passo é fundamental pois, foi possível encontrar um modelo de roteador que cumpra com as especificações recomendadas pelos desenvolvedores do *OpenWRT* por meio dele. Esta configuração descrita em (OpenWrt Project., 2025) consiste em: no mínimo 8MB de memória *flash* para acomodar o *firmware*, sendo 16MB a quantidade recomendada, e no mínimo 64MB de memória *RAM* (*Random Access Memory*), sendo 128MB a quantidade recomendada.

Uma vez que o *hardware* recomendado foi definido, acessa-se a tabela de *hardware* suportado disponível em (OpenWrt Project., 2025), logo em seguida é aplicado um filtro nas colunas de memória *RAM* e memória *flash* a fim de os requisitos recomendados serem atendidos. Inicialmente muitas opções que atendem a configuração recomendada surgem, dentre estas várias opções, o modelo *Archer C6* da marca *TP-Link* é o escolhido. Isto ocorreu em razão do seu suporte ao método de instalação "*OEM easy installation*", que facilita o processo de substituição do *firmware*, e à qualidade da documentação disponível para tal modelo de roteador.

Assim que tal modelo foi adquirido, a imagem do *firmware* disponibilizada na página "OpenWrt - Archer C6 v3" (OpenWrt Project..., 2024) é baixada em um computador que dispõe de interface de rede cabeada, logo em seguida o roteador é ligado à energia, e conectado via enlace cabeado ao computador. Este computador é utilizado para todo o processo de substituição do *firmware* e configuração inicial do roteador, isto deve ser feito antes do dispositivo ser inserido na rede local doméstica. Uma vez que o roteador foi ligado e conectado ao computador em posse da imagem do OpenWrt, acessa-se a página de administrador padrão da *TP-Link* por meio do IP 192.168.0.1, nesta página é realizado o login como administrador com as credenciais padrão, e logo em seguida o passo a passo descrito na seção "*OEM easy installation*" de (OpenWrt Project..., 2024) é seguido. Ao fim deste processo de instalação o roteador é reiniciado por conta própria já dispondo da interface *web* LuCI do OpenWrt para que a configuração inicial do roteador seja realizada.

4.1.2 Ajustes na Topologia da Rede

Para que a captura de pacotes seja possível, dois elementos são fundamentais na rede doméstica, um roteador que disponha da funcionalidade de *port-mirroring* e um servidor *sniffer* conectado à porta alvo do *port-mirroring*. Entretanto, antes de estes dispositivos serem incluídos à rede doméstica, alguns ajustes na topologia desta podem ser necessários. No Brasil é uma prática comum, os ISPs (*Internet Service Providers*) oferecerem junto do pacote de banda larga um dispositivo *modem* que serve também de roteador, entretanto estes dispositivos raramente contemplam a funcionalidade de *port-mirroring* nativamente, e a substituição de seu *firmware* padrão não é aconselhável pelos ISP's.

Sendo assim, a topologia proposta consiste em negligenciar o *modem* fornecido pelo ISP enquanto agente centralizador da rede, isto é feito por meio da centralização da rede doméstica no roteador com *port-mirroring*. Para isso, conecta-se o roteador que dispõe da funcionalidade de *port-mirroring* ao *modem* disponibilizado pelo ISP por meio de qualquer interface do enlace cabeado deste. Após isso, conecta-se o servidor *sniffer* à porta destino do *port-mirroring*, de forma que todo o tráfego espelhado da interface *bridge* do roteador chegue ao servidor *sniffer*. Feito isso, o roteador é configurado como ponto de acesso *Wi-Fi* e são conectados à ele os dispositivos da rede, seja pelas interfaces disponíveis no enlace cabeado ou pelo ponto de acesso *Wi-Fi*. Ao fim deste processo, a rede doméstica deve ter uma topologia semelhante à descrita na Figura 2.

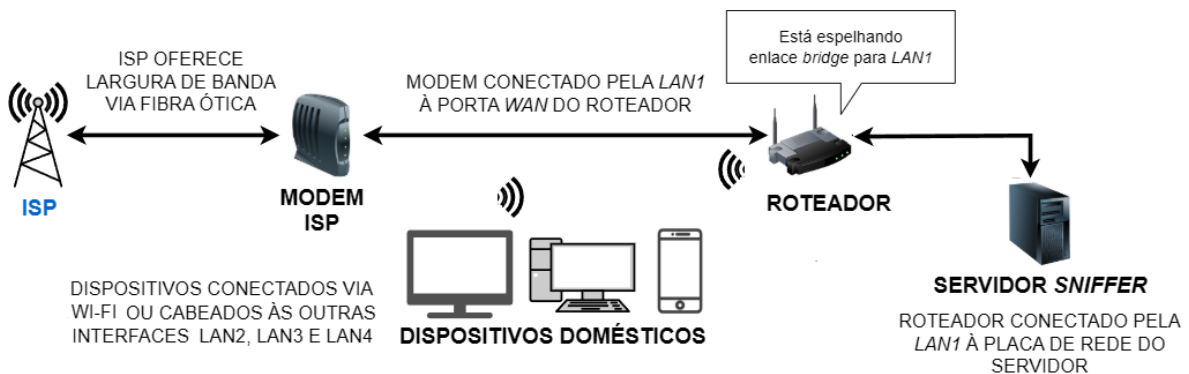


Figura 2 – Topologia genérica proposta para a rede doméstica. Fonte: Do Autor.

Embora a rede dessa forma já atenda à demanda do trabalho, alguns ajustes foram realizados antes dessa topologia ser concretizada, para que tudo funcione apropriadamente. Nesta seção são apresentados superficialmente alguns desses ajustes, mas cada ajuste é detalhado nas próximas seções já considerando o ambiente prático de desenvolvimento do trabalho. Mapeia-se a rede para entender quais endereços IP foram atribuídos ao roteador e ao *modem*. Isso é necessário para prevenir que esses dispositivos compartilhem o mesmo endereço IP, uma vez que isso pode gerar conflitos no roteamento de pacotes. Caso os dois dispositivos tenham o mesmo endereço IP, é aconselhável alterar o endereço

IP do roteador que suporta *port-mirroring*, pois comumente os *modems* disponibilizados pelo ISP possuem travas a nível de *firmware* que impedem mudanças como essas.

Outro ajuste pode ser necessário, este consiste em escolher um endereço IP fixo para o *sniffer*, desabilitar em suas configurações a opção de DHCP (*Dynamic Host Configuration Protocol*), configurar o *gateway* padrão do servidor como o endereço IP do roteador e reservar o endereço IP que é usado pelo *sniffer* nas configurações do roteador. Este último passo deve ser feito a fim de que o endereço IP fixado para o *sniffer* não seja atribuído a outro dispositivo eventualmente, tornando mais fácil de localizá-lo na rede local. Ao fim, teremos algo como o exemplo descrito na Figura 3.

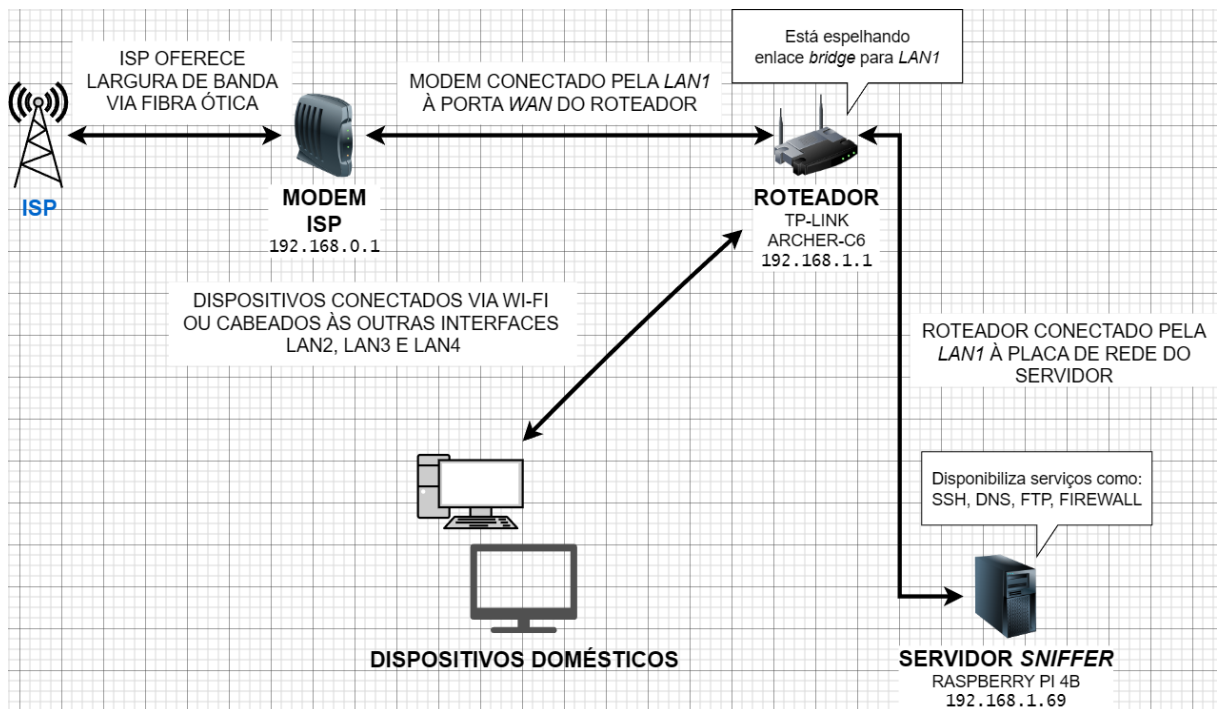


Figura 3 – Topologia proposta para a rede doméstica detalhada. Fonte: Do Autor.

Na Figura 3 são especificados quais dispositivos são utilizados para cumprir os papéis genéricos explicitados em Figura 2. Mas o mais importante da Figura 3 é entender como os ajustes citados anteriormente foram realizados. Note que o *modem* possui IP fixo 192.168.0.1, foi atribuído ao roteador o IP fixo 192.168.1.1 e o *gateway* da interface cabeada foi configurado como 192.168.0.1, que é o IP fixo do *modem*, ao servidor *sniffer* foi atribuído e devidamente configurado o IP fixo 192.168.1.69, o qual também foi reservado no roteador. Desta forma todos os requisitos e cuidados necessários para o funcionamento da topologia genérica descrita anteriormente são cumpridos.

4.1.3 Configurando o Roteador

Uma vez que a imagem do *OpenWrt* foi instalada no roteador, este foi configurado para atender os requisitos práticos do trabalho. Nesta seção são abordadas quais configu-

rações foram realizadas. No primeiro passo conecta-se um computador via enlace cabeado ao roteador, tal qual na Figura 2, e realiza-se o *login* como administrador por meio da interface web LuCI disponibilizada em 192.168.1.1. Uma vez que o *login* foi realizado, o *login* e senha de administrador foram alterados e o *login* via SSH foi habilitado para interface cabeada, tudo isso é realizado por meio da página inicial da LuCI.

Uma vez que o SSH foi habilitado, estabelece-se a conexão por meio de um cliente SSH e das credenciais de *login* configuradas previamente. Uma vez que a sessão SSH foi estabelecida, realiza-se a atualização dos pacotes nativos e instalação do pacote *port-mirroring* (MARAYA, 2024) via Opkg (OpenWrt Project..., 2024b), que é o gerenciador de pacotes do OpenWrt. Assim que o pacote de *port-mirroring* foi instalado, este é configurado por meio do arquivo próprio de configuração. Para configurar tal pacote é necessário conhecer a lista das interfaces disponíveis no roteador e seus respectivos nomes. A Figura 4 mostra o comando utilizado a fim de listar as interfaces.

```
root@OpenWrt:~# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1504 qdisc mq state UP qlen 1000
    link/ether b0:a7:b9:9e:85:43 brd ff:ff:ff:ff:ff:ff
3: wan@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether b0:a7:b9:9e:85:43 brd ff:ff:ff:ff:ff:ff
4: lan1@eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue master br-lan state LOWERLAYERDOWN
    qlen 1000
    link/ether b0:a7:b9:9e:85:43 brd ff:ff:ff:ff:ff:ff
5: lan2@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-lan state UP qlen 1000
    link/ether b0:a7:b9:9e:85:43 brd ff:ff:ff:ff:ff:ff
6: lan3@eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue master br-lan state LOWERLAYERDOWN
    qlen 1000
    link/ether b0:a7:b9:9e:85:43 brd ff:ff:ff:ff:ff:ff
7: lan4@eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue master br-lan state LOWERLAYERDOWN
    qlen 1000
    link/ether b0:a7:b9:9e:85:43 brd ff:ff:ff:ff:ff:ff
8: br-lan: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether b0:a7:b9:9e:85:43 brd ff:ff:ff:ff:ff:ff
9: phy1-ap0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-lan state UP qlen 1000
    link/ether b0:a7:b9:9e:85:45 brd ff:ff:ff:ff:ff:ff
```

Figura 4 – Interfaces de rede disponíveis no roteador Archer C6 v3. Fonte: Do Autor.

Uma vez que obtém-se o nome de todas interfaces do roteador, a configuração do pacote de *port-mirroring* foi realizada. Modifica-se a configuração padrão, disponibilizada na Figura 5, a fim de que o tráfego da interface *bridge br-lan* seja espelhado para a interface cabeada alvo *lan1*. Ao fim deste processo obtém-se o arquivo de configuração mostrado na Figura 6. Neste arquivo o campo *option source_ports* foi alterado para *br-lan*, que é o nome da interface *bridge* da qual queremos espelhar os pacotes, e o campo *option target* para *lan1*, que é o nome da interface para qual queremos direcionar os pacotes espelhados.

Uma vez configurado o pacote de *port-mirroring*, existem alguns comandos fundamentais para iniciar a aplicação, disponíveis no repositório do pacote (MARAYA, 2024) e mostrados no Algoritmo 1.

```

#
# OpenWrt Unified Configuration Interface (UCI) for port-mirroring
#
# https://github.com/mmaraya/port-mirroring
#

config 'port-mirroring'
    option source_ports 'eth0,wlan0'      # interfaces (maximum of 4) to copy packets from
    option promiscuous '1'                # put source interface(s) in promiscuous mode
    option target '10.1.4.2'              # interface or IP address to send packets to
    option protocol 'TEE'                  # 'TEE' iptables (default) or 'TZSP' TaZmen Sniffer Protocol
    option filter ''                       # optional tcpdump/libpcap packet filter expressions

```

Figura 5 – Configuração padrão *port-mirroring*. Fonte: Do Autor.

```

#
# OpenWrt Unified Configuration Interface (UCI) for port-mirroring
#
# https://github.com/mmaraya/port-mirroring
#

config 'port-mirroring'
    option source_ports 'br-lan'          # interfaces (maximum of 4) to copy packets from
    option promiscuous '1'                # put source interface(s) in promiscuous mode
    option target 'lan1'                  # interface or IP address to send packets to
    option protocol 'TEE'                  # 'TEE' iptables (default) or 'TZSP' TaZmen Sniffer Protocol
    option filter ''                       # optional tcpdump/libpcap packet filter expressions

```

Figura 6 – Configuração modificada *port-mirroring*. Fonte: Do Autor.

Algorithm 1 Comandos importantes port-mirroring. Fonte: Do Autor

```

1  # iniciar port-mirroring em segundo plano com debugger ativo
2  ~$ port-mirroring --debug
3
4  # iniciar port-mirroring como daemon
5  ~$ /etc/init.d/port_mirroring start
6
7  # encerrar daemon do port-mirroring
8  ~$ /etc/init.d/port_mirroring stop
9
10 # ler logs de sistema do port-mirroring
11 ~$ logread | grep port-mirroring

```

Além dessa configuração do pacote de *port-mirroring*, o roteador foi configurado como ponto de acesso *Wi-Fi* por meio da interface web, seguindo o passo-a-passo da página “*Enabling a Wi-Fi access point on OpenWrt*” ([OpenWrt Project...., 2024a](#)). Assim, finalmente, conecta-se o roteador ao restante da rede doméstica, ao conectar o roteador ao *modem* do ISP por meio do enlace cabeado.

4.2 Configurando Servidor *Sniffer*

Com o roteador previamente configurado para o espelhamento de porta (*Port Mirroring*), esta seção detalha os passos de configuração do servidor de captura. O objetivo

foi torná-lo acessível, de gerenciamento simplificado e apto a registrar o tráfego de rede.

4.2.1 Escolha do Servidor

O computador selecionado para atuar como servidor *sniffer* é o *Raspberry Pi 4B* ([Raspberry Pi, 2025](#)) com o sistema operacional *Ubuntu Server 20.04 LTS (Long Term Support)* ([Ubuntu Releases, 2025](#)). A escolha deste computador para servidor baseia-se em três critérios principais: i) seu poder de processamento é mais do que suficiente para a tarefa; ii) seu baixo consumo energético e sua padronização de *hardware*; e iii) fator que garante a alta replicabilidade da metodologia. A versão *Ubuntu Server 20.04 LTS* é escolhida devido à sua elevada popularidade, disponibilidade, estabilidade, ao seu baixo consumo de recursos computacionais e ao seu amplo conjunto de pacotes nativos.

Vale ressaltar que a metodologia descrita nesta seção pode ser replicada em praticamente qualquer computador funcional moderno que possua uma interface de rede e disponha de uma distribuição *Linux* instalada. Entretanto, talvez alguns ajustes tenham que ser realizados a fim de se obter resultados similares aos descritos nas próximas seções. A imagem *Linux* é instalada no servidor por meio do passo a passo descrito em "*Getting started with your Raspberry Pi - Install an operating system*" ([Raspberry Pi., 2025](#)). Por fim, este é conectado à porta *target* (LAN1) do roteador com *firmware* modificado.

4.2.2 Configuração do SSH, FTP e Outros Serviços

A fim de tornar o servidor *sniffer* acessível e de fácil manejo, algumas ferramentas são instaladas e configuradas, e cada uma dessas ferramentas cumpre um papel específico. O *SSH (Secure Shell Protocol)* é habilitado e configurado antes da instalação da imagem *Linux* pela interface *Raspberry Pi Imager*, de forma que, ao o servidor reiniciar após a instalação do sistema operacional, já é possível iniciar uma sessão remota a partir de um cliente conectado à mesma rede local. O *SSH* garante a disponibilidade de acesso, isto é, qualquer dispositivo que disponha de um cliente *SSH* conectado à mesma rede do servidor *sniffer* pode ser usado para acesso à *CLI (Command Line Interface)* deste.

A seguir, é designado um endereço IP estático para o servidor a fim de facilitar o acesso ao dispositivo na rede, de acordo com o passo a passo descrito em "*Setting a Static Ip Address in Ubuntu 20.04 via the Command Line*" ([Linux Config Org, 2024](#)). Também é configurado um servidor *SFTP (Secure File Transfer Protocol)* utilizando a ferramenta *vsftpd* ([Evans. Chris, 2021](#)) para que o processo de retirar e adicionar arquivos ao servidor por meio de um dispositivo externo seja realizado via rede de uma maneira segura. Esta configuração do servidor *SFTP* é realizada seguindo o material "*vsftpd - Community Help Wiki*" ([Ubuntu Community., 2015](#)), e adiciona muita praticidade para o acesso aos arquivos de captura gerados pelo *sniffer*, possibilitando a visualização dos fluxos

de pacotes capturados como planilhas em outros dispositivos. Também é configurado o *firewall* UFW (*Uncomplicated Firewall*) ([Ubuntu Community, 2023](#)), que é nativo do Ubuntu, a fim de restringir com rapidez o acesso aos serviços do servidor.

Por fim, a ferramenta DuckDNS é configurada de acordo com o passo a passo descrito em "Duck DNS - Install" ([DuckDNS Org, 2025](#)), a fim de possibilitar o acesso ao servidor de maneira externa à rede local por meio de um nome de domínio customizado. Para que os serviços disponibilizados pelo servidor sejam acessíveis de fora da rede doméstica, é necessário encontrar o endereço IP do *sniffer* relativo ao *modem* do ISP e habilitar o *Port Forwarding* para as portas do SSH e SFTP; para isso, utiliza-se a ferramenta *Nmap* ([Nmap Org, 2025](#)). Contempladas estas etapas, torna-se possível acessar o servidor de qualquer rede, possibilitando o trabalho de maneira remota.

4.2.3 Configurando *Sniffer*

Antes de partir para a escolha de um *sniffer* para a captura de pacotes de rede, é importante lembrar qual é o principal objetivo do trabalho, uma vez que ele impacta diretamente na escolha dessa ferramenta. Dito isso, de maneira breve, os principais objetivos do trabalho são, definir o passo a passo para a construção de um *dataset* de fluxo de pacotes para sistemas IDS (*Intrusion Detection Systems*) e ADS (*Anomaly Detection Systems*) com base em tráfego doméstico e construir um *dataset* seguindo o passo-a-passo.

Considerando esses objetivos, alguns questionamentos surgem, com destaque para: i) como capturar todo o tráfego que permeia a rede doméstica? ii) quais *features* esse *dataset* deve possuir? iii) uma vez definidas as *features*, como obtê-las? iv) como agrupar o alto volume de pacotes a fim de obter um *dataset* conciso sem perda de informações valiosas para os sistemas IDS e ADS? Nota-se que esses são questionamentos essenciais para que o trabalho possa chegar a algum resultado plausível. Observa-se também que algumas dessas perguntas são respondidas: i) todo o tráfego que permeia a rede doméstica pode ser capturado por meio de mudanças na topologia da rede, uso de um roteador com *port-mirroring* e de um servidor *sniffer*; ii) parte-se do princípio de que esse *dataset* deve ter as mesmas *features* que *datasets* renomados como KDD-99 ([ARCHIVE, 1999](#)), CIC-IDS2017 ([CYBERSECURITY, 2017](#)) e UNSW-NB15 ([MOUSTAFA, N.; SLAY, J., 2015](#)). As perguntas em aberto são respondidas ainda nesta seção, com o detalhamento devido.

A fim de responder às perguntas que se mantiveram em aberto, deve-se entender primeiro quais *sniffers* de pacotes existem, quais são suas funcionalidades e limitações. O primeiro *sniffer*, que constitui uma solução *naive* quando se deseja capturar pacotes de uma interface de rede, é o *tshark* ([Tshark, 2024](#)). Esta ferramenta CLI (*Command Line Interface*) oferece algumas funcionalidades padrões como captura de tráfego de pacotes para formato binário PCAP (*Packet Capture*) e leitura de arquivos PCAP para um formato *human readable* modificável. Outra ferramenta tão importante quanto é o OpenArgus

(OPENARGUS, 2024). Este é um analisador e gerador de fluxo de pacotes, capaz de capturar um fluxo de pacotes e salvá-lo em um arquivo binário ARGUS. Esta ferramenta também é capaz de transformar um arquivo ARGUS em uma mensagem *human readable*. Uma terceira ferramenta é o *CICFlowmeter* (Lashkari, Arash Habibi, 2016a). Este também é um analisador e gerador de fluxo de pacotes, capaz de capturar um fluxo de pacotes e salvá-lo diretamente em um arquivo CSV (*Comma Separated Values*) com as mesmas 80 *features* base do CIC-IDS2017 (CYBERSECURITY, 2017) e de *datasets* semelhantes, como CIC-AAGM2017 (*Android Adware and General Malware Dataset*) (LASHKARI et al., 2017) e CIC-AndMal2017 (*Android Malware Dataset*) (LASHKARI et al., 2018).

Para determinar o *sniffer* a ser utilizado desenvolveu-se um pequeno *benchmark* para comparação, para isso o *port-mirroring* foi inicializado como *daemon* e 500 pacotes da rede doméstica foram capturados utilizando o *tshark* a fim de gerar um arquivo PCAP, para isso o seguinte comando foi utilizado:

Algorithm 2 Geração de *sample.pcap* para *benchmark* dos sniffers. Fonte: Do Autor

```
1 # capturar tráfego da interface eth0 e gerar o arquivo sample.pcap
2 ~$ tshark -i eth0 -c 500 -w sample.pcap
```

Este arquivo PCAP foi então utilizado nos três *sniffers* para permitir a geração do *dataset*. O primeiro a ser testado foi o próprio *tshark*, para isso o seguinte comando foi executado:

Algorithm 3 Geração de *tshark_sample.csv* a partir do *sample.pcap* via modo de leitura do *tshark*. Fonte: Do Autor

```
1  # coleta de campos do pcap para gerar um .csv via modo de leitura do tshark
2  ~$ tshark -r sample.pcap -T fields \
3      -e frame.number \
4      -e frame.time_epoch \
5      -e frame.len \
6      -e ip.src \
7      -e ip.dst \
8      -e ip.proto \
9      -e ip.len \
10     -e ip.ttl \
11     -e tcp.srcport \
12     -e tcp.dstport \
13     -e tcp.len \
14     -e tcp.flags \
15     -e tcp.window_size_value \
16     -e tcp.analysis.lost_segment \
17     -e tcp.analysis.retransmission \
18     -e udp.srcport \
19     -e udp.dstport \
20     -e udp.length \
21     -e http.request.method \
22     -e http.host \
23     -e dns.qry.name \
24     -e tcp.seq \
25     -e tcp.ack \
26     -e tcp.time_relative \
27     -e tcp.analysis.ack_rtt \
28     -e http.content_length \
29     -e ftp.request.command \
30     -e ftp.response.code
31     -E header=y -E separator=, -E quote=d > tshark_sample.csv
```

Observa-se que este comando é relativamente simples: o *tshark* está sendo executado em modo de leitura, evidenciado pelo uso da opção *-r* (de *read*), para a leitura do conteúdo de *sample.pcap*. Em seguida, a opção *-T* (de *transform*) é utilizada para especificar o formato da saída do comando; neste caso, é especificado o formato tabular por meio do argumento *fields*. Posteriormente, a opção *-e* (de *extract*) é empregada para especificar quais campos devem aparecer no resultado do comando. Ao final da especificação dos campos, a opção *-E* é utilizada para definir a posição do cabeçalho da tabela, o separador dos atributos (neste caso, a vírgula) e o estilo das aspas (aspas duplas). Por fim, é realizado o redirecionamento da saída do comando para um arquivo denominado *tshark_sample.csv*.

Dessa forma, este comando aplicado ao arquivo *sample.pcap* gera um arquivo *tshark_sample.csv*, que está representado a seguir em formato de tabela:

frame.number	frame.time_epoch	frame.len	...	tcp.flags
1	1.751.244.749.609.210.000	60	...	0x0010
2	1.751.244.749.834.420.000	118	...	0x0018
3	1.751.244.749.892.320.000	60	...	0x0010
4	1.751.244.749.907.700.000	68	...	
5	1.751.244.750.175.450.000	60	...	
...	...	...	...	...
496	1.751.244.805.699.690.000	60	...	0x0010
497	1.751.244.805.892.400.000	118	...	0x0018
498	1.751.244.805.924.280.000	68	...	
499	1.751.244.805.950.670.000	60	...	0x0010
500	1.751.244.806.148.430.000	118	...	0x0018

Tabela 1 – Representação de *tshark_sample.csv* como tabela. Fonte: Do Autor

A Tabela 1 possui apenas as 21 *features*. Estas são evidenciadas no Algoritmo 3, o que não constitui um número suficiente de *features* quando comparado com *datasets* de referência como CIC-IDS2017, que possui 84 *features*. Além disso, é importante notar que o modo de leitura do *tshark* compromete alguns atributos ao ocultar seus valores em determinadas linhas. O problema central dessa abordagem é que tal omissão de dados não é justificada nem explicada em documentação alguma, gerando atributos esparsos no *dataset*, o que demandaria tratamento em uma etapa posterior de formatação.

Além disso, o *dataset* construído pelo modo de leitura do *tshark* possui uma linha para cada pacote coletado, o que definitivamente não é ideal, uma vez que o fluxo de pacotes em um curto período de tempo costuma ter milhares e milhares de pacotes, ocasionando um *dataset* com muitas amostras. Tal problema pode ser solucionado por meio do agrupamento de pacotes em um *packet flow*, ou fluxo de pacotes, que consiste basicamente em agrupar amostras similares referentes a uma mesma conexão. São com esses agrupamentos que ferramentas como *OpenArgus* e *CICFlowmeter* trabalham. Todos esses fatores mencionados colaboram para a inviabilidade do *tshark* na criação de uma versão inicial do *dataset*.

Na sequência, a saída proporcionada pelo *OpenArgus* ao ler o arquivo *sample.pcap* é abordada. Vale ressaltar, de maneira prévia ao resultado do *benchmark*, que o *OpenArgus* apresenta uma limitação: este não é capaz de exportar um arquivo CSV, nem por meio de soluções alternativas como foi feito com o *tshark*, sendo capaz apenas de gerar mensagens legíveis por humanos (*human readable messages*) com pouca customização. Dito isso, a fim de fazer o *OpenArgus* ler o arquivo *sample.pcap* e gerar essa mensagem legível, os seguintes comandos foram executados:

Algorithm 4 Geração de *argus_sample.csv* a partir do *sample.pcap* via modo de leitura do *OpenArgus*. Fonte: Do Autor

```

1 # leitura de sample.pcap pelo OpenArgus e escrita em um .argus
2 ~$ sudo argus -r sample.pcap -w sample.argus
3 ~$ sudo ra -r sample.argus -u > argus_sample.csv

```

No Algoritmo 4 o primeiro comando chama o executável `argus` para ler do arquivo `sample.pcap`, realizar a análise do fluxo de pacotes, e escrever o arquivo binário `sample.argus`. O comando seguinte evoca o executável `ra` responsável por ler o arquivo `sample.argus` e transformá-lo em uma mensagem legível à qual é salva por indireção no arquivo `argus_sample.csv`, o qual dispõe do seguinte conteúdo:

StartTime	Flgs	Proto	SrcAddr	Sport	Dir	DstAddr	Dport	TotPkts	TotBytes	State
1753755015.539156		man	0	0		0	0	0	0	STA
1751244749.609213	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	37	37	3264	CON
1751244749.907707	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	2	2	136	INT
1751244750.175450	e	arp	192.168.0.8	who	->	192.168.0.69	2	2	102	CON
1751244750.219507	e	udp	192.168.0.8.mdns	->	->	224.0.0.251.mdns	8	8	1136	INT
1751244750.219740	e	udp	fe80::cbb7:210b:6*.mdns	->	->	ff02::fb.mdns	8	8	1296	INT
1751244750.627766	e	ipv6*	fe80::2235:43ff:f*.0x0085	->	->	ff02::1.0x0000	1	1	182	NRS
1751244750.631277	e	ipv6*	fe80::dea6:32ff:f*.0x008f	->	->	ff02::16.0x0000	2	2	260	MHR
1751244754.750450	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244755.850256	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	3	3	204	REQ
1751244759.870169	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244763.837440	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	3	3	204	REQ
1751244764.554466	e	ipv6*	fe80::2235:43ff:f*.0x0085	->	->	ff02::1.0x0000	1	1	182	NRS
1751244764.558238	e	ipv6*	fe80::dea6:32ff:f*.0x008f	->	->	ff02::16.0x0000	2	2	260	MHR
1751244764.996210	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244769.879004	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	3	3	204	REQ
1751244770.102601	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244775.233884	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244775.920815	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	3	3	204	REQ
1751244778.071302	e	ipv6*	fe80::2235:43ff:f*.0x0085	->	->	ff02::1.0x0000	1	1	182	NRS
1751244778.075235	e	ipv6*	fe80::dea6:32ff:f*.0x008f	->	->	ff02::16.0x0000	2	2	260	MHR
1751244780.345534	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244783.907971	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	3	3	204	REQ
1751244785.465252	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244789.949711	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	2	2	136	REQ
1751244790.584119	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244791.588367	e	ipv6*	fe80::2235:43ff:f*.0x0085	->	->	ff02::1.0x0000	1	1	182	NRS
1751244791.592234	e	ipv6*	fe80::dea6:32ff:f*.0x008f	->	->	ff02::16.0x0000	2	2	260	MHR
1751244795.704962	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244795.889009	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	3	3	204	REQ
1751244796.680066	e	arp	192.168.0.8	who	->	192.168.0.69	2	2	102	CON
1751244800.836255	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	40	40	3560	CON
1751244801.828176	*	llc	01:80:c2:00:00:00.0x20	<->	->	20:35:43:12:02:90.0x35	3	3	204	REQ
1751244804.490842	e	ipv6*	fe80::2235:43ff:f*.0x0085	->	->	ff02::1.0x0000	1	1	182	NRS
1751244804.495242	e	ipv6*	fe80::dea6:32ff:f*.0x008f	->	->	ff02::16.0x0000	2	2	260	MHR
1751244805.950674	e	tcp	192.168.0.8.52389	<?>	->	192.168.0.69.ssh	3	3	296	CON
1753755016.553831		man	0	0		19	1	0	10963180	STP

Figura 7 – Conteúdo de `argus_sample.csv`. Fonte: Do Autor

Observe que na Figura 7 temos de fato uma mensagem legível. Entretanto, dispomos de poucas *features* quando comparado ao resultado do *tshark* e a outros *datasets*. Além disso, a estrutura do arquivo é distinta da estrutura de um arquivo CSV, o que cria a necessidade de formatar os dados adequadamente. Contudo, é importante notar que a análise de fluxo de pacotes efetivamente agrupou os 500 pacotes capturados, de forma que agora temos apenas 37 linhas que representam nosso conjunto.

Em seguida, a saída proporcionada pelo *CICFlowmeter* ao ler o arquivo `sample.pcap` é analisada. Vale ressaltar que a ferramenta *CICFlowmeter* possui um amplo conjunto de funcionalidades, desde a leitura de arquivos PCAP para geração de arquivos CSV locais até mesmo captura de tráfego no nível de pacote e transmissão dos pacotes capturados agregados para servidores HTTP (*Hypertext Transfer Protocol*), dentre outras funcionalidades. Além disso, existem duas versões do *CICFlowmeter*, a versão original escrita em Java conhecida como *ISCXFlowMeter* (Lashkari, Arash Habibi, 2016b) e uma versão paralela escrita em Python (Nguyen, Dat-Thinh, 2022) que conta com as mesmas funcionalidades. A versão escrita em Python é escolhida em detrimento da facilidade de *setup* da ferramenta para execução no servidor *sniffer*. Realizada a instalação da ferramenta por meio do guia disponível no repositório, o Algoritmo 5 é executado.

Algorithm 5 Geração de *cicflowmeter_sample.csv* a partir do *sample.pcap* via modo offline do CICFlowmeter Python. Fonte: Do Autor

```

1 # leitura de sample.pcap pelo CICFlowmeter (python)
2 ~$ sudo cicflowmeter -f sample.pcap -c cicflowmeter_sample.csv

```

Uma vez que o comando descrito em Algoritmo 5 é executado, ele invoca o executável da aplicação passando uma *tag -f* que indica que é realizada a leitura de um arquivo PCAP, e uma *tag -c* que indica que a saída deve ser um arquivo CSV com o nome que vem logo à seguir. Ao fim da execução do comando, o arquivo *cicflowmeter_sample.csv* é criado com as mesmas 80 *features* base do CICIDS-2017 e com somente 2 linhas, em razão da agregação aplicada aos pacotes.

src_ip	dst_ip	src_port	dst_port	...	subflow_bwd_byts
192.168.0.8	192.168.0.69	52389	22	...	25960
192.168.0.8	224.0.0.251	5353	5353	...	0

Tabela 2 – Representação de *cicflowmeter_sample.csv* como tabela. Fonte: Do Autor

Como demonstrado na Tabela 2, os 500 pacotes capturados em *sample.cap* foram reduzidos a 2 linhas, sendo a primeira linha a agregação do fluxo de pacotes da conexão SSH de um computador pessoal com o servidor *sniffer*, evidenciado pelo atributo *dst_port* que possui valor 22 para esta linha, que é a porta padrão para conexões SSH, já a segunda linha diz respeito a uma conexão do mesmo computador pessoal com um servidor DNS. Tendo em vista que este computador pessoal é o único dispositivo conectado ao *sniffer* durante a captura, este foi um bom agrupamento de pacotes, pois é capaz de representar o fluxo de pacotes da rede de uma maneira concisa e objetiva. Assim, as três principais ferramentas, Tshark, OpenArgus e *CICFlowmeter* foram devidamente comparadas entre si, com base nos resultados observados, o *CICFlowmeter* é o escolhido. Isso se dá pois, o uso do *CICFlowmeter* descarta a necessidade de codificação de uma ferramenta para agregação de pacotes ao aproximar a estrutura do *dataset* doméstico em construção ao do *dataset* de referência CIC-IDS2017 (CYBERSECURITY, 2017) e de *datasets* da mesma família. Isso ocorre pois, tal ferramenta garante a mesma quantidade de *features* base no *dataset*, além de agrupar os pacotes da mesma maneira.

Com isso, encerra-se a etapa de configuração do *sniffer*. Entretanto, ao comparar os resultados obtidos pelo *CICFlowmeter* com os *datasets* de referência da família CIC de *datasets* surgem algumas perguntas: i) como classificar um fluxo de pacotes como benigno ou malicioso? ii) é possível gerar artificialmente tráfego benigno e malicioso numa rede doméstica? Como fazer isto? Estas perguntas são respondidas na Seção 4.3.

4.3 Geração de Tráfego

Para dar continuidade ao trabalho, é fundamental validar se a arquitetura de rede proposta atende às necessidades do trabalho antes da coleta real de dados em ambiente doméstico para construção do *dataset*. Isto é, torna-se necessário validar se a arquitetura e as etapas de configuração previamente definidas permitem a captura de pacotes por meio do *sniffer* e a construção adequada de fluxos de pacotes que façam sentido.

Para essa validação, é essencial comparar o tráfego da rede com o arquivo CSV gerado pelo *CICFlowmeter*. Entretanto, esta tarefa pode se tornar complexa em um cenário com dispositivos reais conectados à rede, em detrimento do número de informações trocados pelos dispositivos. Uma possível abordagem é testar a captura de tráfego com poucos dispositivos conectados, utilizando-os de maneira controlada durante a captura. Contudo, tal procedimento seria manual e impraticável para longas capturas.

Uma solução alternativa consiste na geração de tráfego que simula o comportamento esperado em uma rede doméstica. Isso é possível por meio da estruturação de um ambiente controlado, no qual é possível definir padrões específicos de comportamento no tráfego gerado. A definição desses comportamentos bem delimitados facilita a validação da arquitetura e das configurações realizadas previamente, uma vez que permite a comparação direta entre o tráfego da rede e o CSV gerado, tendo em vista que a saída esperada é conhecida, visto que foi bem definida anteriormente.

A geração simulada de tráfego é extremamente vantajosa quando associada à simulação de ataques e intrusões na rede. Ao criar um ambiente controlado com comportamentos de interação entre dispositivos bem definidos, torna-se possível categorizar facilmente no fluxo de dados final gerado pelo *CICFlowmeter* quais registros correspondem a ataques ou intrusões, uma vez que o ambiente foi modelado de forma controlada para os experimentos. Consequentemente, a criação de um *dataset* com rótulos apropriados e precisos torna-se possível, o que beneficia significativamente as etapas de treinamento de modelos de aprendizado de máquina às quais o *dataset* gerado seja submetido.

Nesse contexto, as próximas subseções abordam a estruturação e configuração desse ambiente de simulação de tráfego, que foi construído com o objetivo de validar o funcionamento da arquitetura de rede proposta anteriormente, antes que a captura real de dados da rede para geração do *dataset* fosse feita.

4.3.1 Geração de Tráfego Normal

Uma abordagem direta para gerar tráfego que simula o comportamento natural consiste na construção de *scripts* que acessam *websites* e se comunicam com outros serviços, baseando-se em comportamentos bem definidos. Esta ideia se baseia no princípio de que, em uma rede doméstica, é possível categorizar os usuários pela forma como interagem

com a tecnologia. Dessa forma, pode-se encontrar em uma rede doméstica desde usuários que utilizam a tecnologia de maneira mais intensiva, acessando servidores FTP, realizando *download* e *upload* de arquivos grandes, ou navegando na Internet durante a maior parte do dia, até usuários que fazem apenas buscas esporádicas na Internet e acessam serviços de *streaming* de forma rotineira. Além disso, é natural imaginar a presença de dispositivos IoT que enviam e recebem métricas continuamente ao longo do dia.

Considerando a arquitetura apresentada anteriormente, para simular tráfego de maneira simplificada, seria suficiente conectar novos dispositivos à rede do roteador com OpenWRT e fazer com que estes dispositivos executem os *scripts* que simulam os comportamentos dos usuários. No entanto, é possível realizar estes testes utilizando o próprio dispositivo *sniffer*, ou até mesmo apenas um computador pessoal, o que facilita a orquestração de processos e torna o ambiente de teste mais simples e replicável.

Para tal implementação, é necessário o entendimento sobre Contêineres *Docker*, que são ambientes de virtualização sem *Hypervisor*. Um *Hypervisor* é um processo responsável pela alocação de recursos físicos do *hardware* hospedeiro para o sistema operacional da máquina virtual. Os Contêineres *Docker* fazem uso compartilhado dos recursos do sistema operacional hospedeiro, compartilhando binários, bibliotecas e demais componentes, dispensando um *Hypervisor* para alocação de recursos. Isso permite que um *container* não necessite de um sistema operacional próprio e completo, tornando-os mais leves que máquinas virtuais tradicionais. Dessa forma, os contêineres *Docker* conseguem conter aplicações inteiras, incluindo todas as bibliotecas, configurações, variáveis de ambiente e uma rede virtual própria, sem afetar o funcionamento de outros serviços hospedados no mesmo *hardware*, de maneira isolada e contida. O uso de contêineres para a criação de ambientes de teste isolados é amplamente adotado devido à facilidade de replicação de tais ambientes por meio dos *Dockerfiles* ou pelo utilitário *Docker Compose*. A Figura 8 ilustra a diferença entre um *Container Docker* e uma máquina virtual convencional, auxiliando na compreensão da explicação apresentada.

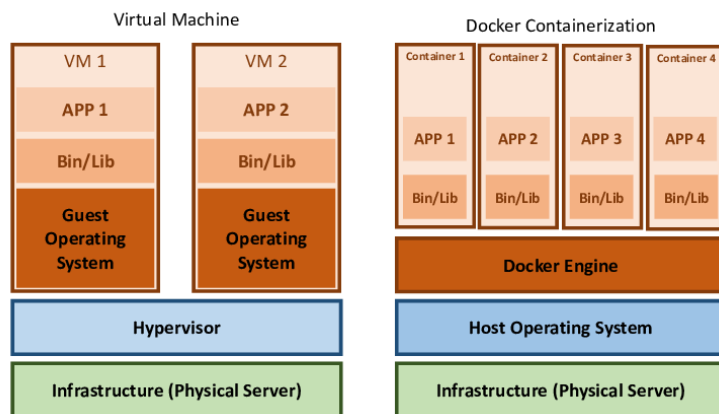


Figura 8 – Comparativo entre Contêineres *Docker* e Máquinas Virtuais. Fonte: (MEHR et al., 2023).

Para consolidar o entendimento, é importante recapitular que, visando construir um ambiente de teste replicável e controlado, utilizam-se Contêineres *Docker*. Para simular tráfego nesse ambiente, empregam-se *scripts* na máquina *host* que descrevem comportamentos pré-definidos de usuários domésticos e comunicam-se com os serviços dos contêineres. Entretanto, surge uma questão: para que os contêineres se comuniquem com processos da máquina *host*, estes necessitam de uma rede virtual *Docker*. Contudo, a rede padrão *Docker bridge* não trata cada *container* como um dispositivo separado, o que seria ideal para a arquitetura definida neste trabalho, uma vez que se busca simular diferentes usuários utilizando dispositivos distintos. Os contêineres nessa rede *bridge* são tratados como processos separados sobre o mesmo *host*. Isso faz com que, caso os contêineres precisem se comunicar com processos da máquina *host*, estes não passem pelo roteador ao qual a máquina *host* está conectada, mas sim pela pilha TCP/IP local.

Para solucionar tal problema, é necessário configurar uma rede *Docker macvlan*, que é uma *virtual local area network* e que tem por função atribuir endereços MAC (*Media Access Control*) únicos a cada *container*. Para tal configuração, é necessário executar os comandos apresentados no Algoritmo 6.

Algorithm 6 Criação de *Docker macvlan* para simular contêineres como diferentes dispositivos físicos. Fonte: Do autor

```

1  # rede virtual docker que simula dispositivos físicos:
2  sudo docker network create -d macvlan \
3    --subnet=192.168.1.0/24 \      # subnet
4    --gateway=192.168.1.1 \      # endereço IP do roteador OpenWRT
5    -o parent=enp7s0 \           # nome da interface de rede da máquina host
6    homelab                      # nome da rede docker

```

O comando *docker network create* é responsável pela criação de uma nova rede *Docker*. A *flag* *-d* especifica o *driver macvlan* a ser utilizado, a *flag* *--subnet* indica o segmento de rede, a *flag* *--gateway* recebe o endereço do roteador OpenWRT, e a *flag* *-o parent* indica a interface de rede física a ser utilizada como base.

Criada a rede virtual *Docker* apropriada, pode-se supor que já é possível os pacotes partirem de um *container* para a máquina *host* pelo roteador, tal qual dispositivos físicos pertencentes à rede fariam. Contudo, isso não ocorre devido a uma medida de segurança - o sistema operacional da máquina *host*, ao identificar um pacote com endereço IP de destino igual ao endereço IP de origem, bloqueia o pacote como medida preventiva contra ataques de *spoofing*.

A fim de solucionar tal problema, é necessário fazer com que esses pacotes possuam endereços IP de origem e destino distintos. Para isso, cria-se uma interface de rede virtual por meio dos comandos descritos no Algoritmo 7, à qual se atribui um endereço IP fixo, diferente do endereço da interface de rede padrão. Desta maneira, os pacotes enviados de

um *container* para a máquina *host* possuem endereços IP de origem e destino diferentes, transitam pelo roteador e retornam à máquina *host* com êxito. Assim, torna-se possível que esse tráfego simulado gerado pelo esquema representado na Figura 9 seja capturado pelo *sniffer* da rede.

Algorithm 7 Criação de Interface de Rede Virtual na máquina *host* Fonte: Do autor

```

1  # interface virtual macvlan-host
2  sudo ip link add macvlan-host link enp7s0 type macvlan mode bridge
3
4  # atribuição de endereço IP
5  sudo ip addr add 192.168.1.200/24 dev macvlan-host
6
7  # "ligando" a interface
8  sudo ip link set macvlan-host up

```

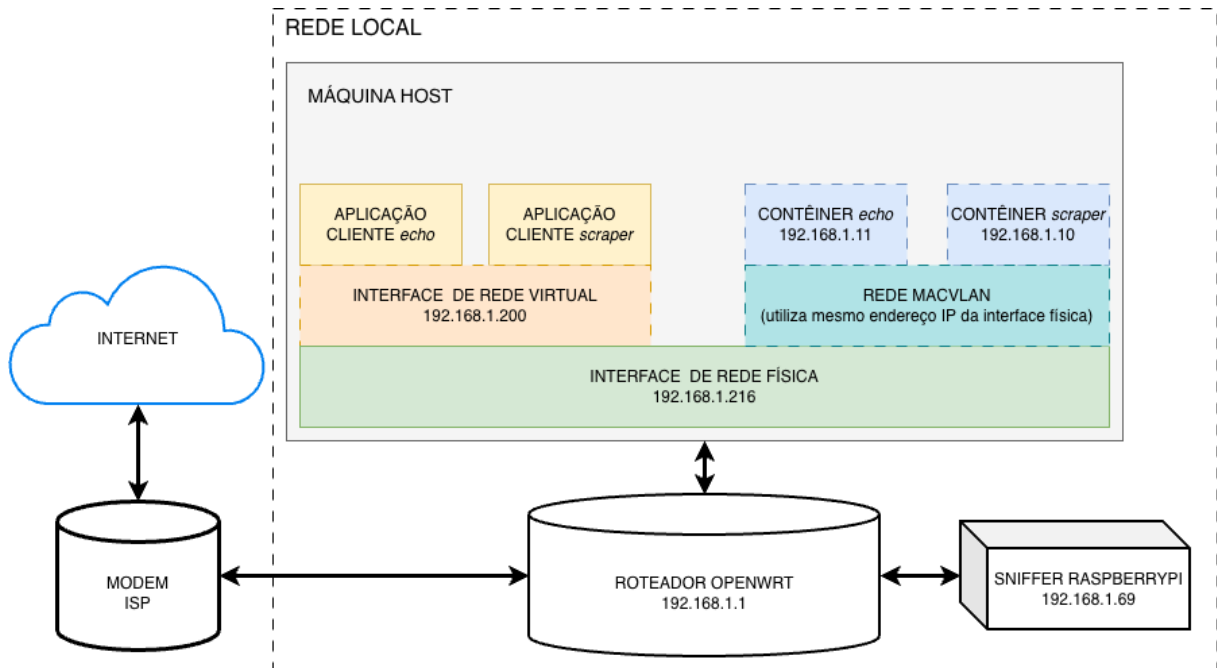


Figura 9 – Rede doméstica com contêineres. Fonte: Do Autor.

Após a realização de todos esses ajustes, realiza-se um teste para validar a arquitetura de rede definida. Para isso, utiliza-se dois contêineres com imagens autorais, cuja criação das imagens e dos contêineres estão descritos no Algoritmo 8.

Algorithm 8 Construção da imagem dos contêineres e execução. Fonte: Do autor

```
1  # construcão do container simple-web-scrapers
2  sudo docker build -t simple-scrapers web_scrapers
3
4  # execucao do container simple-web-scrapers
5  sudo docker run -d \
6      --network homelab \
7      --ip 192.168.1.10 \
8      -p 8001:8001 \
9      --name simple-scrapers \
10     simple-scrapers \
11
12 # construcão do container echo-api
13 sudo docker build -t echo-api echo
14
15 # execucao do container echo-api
16 sudo docker run -d \
17     --network homelab \
18     --ip 192.168.1.11 \
19     -p 8000:8000 \
20     --name echo-api \
21     echo-api \
```

O Algoritmo 8 contém os comandos utilizados para a construção e execução dos contêineres criados para a execução do teste da arquitetura proposta. O primeiro comando cria uma imagem a partir do *Dockerfile* localizado no diretório *web_scrapers* e atribui o nome *simple_scrapers* à imagem criada. O segundo comando inicializa um contêiner com a imagem criada no primeiro comando, atribui este contêiner à rede *macvlan homelab*, mapeia a porta 8001 da máquina *host* à porta 8001 do contêiner, e renomeia este. O terceiro comando cria uma imagem a partir do *Dockerfile* localizado no diretório *echo* e atribui o nome *echo-api* à imagem criada. O quarto comando inicializa um contêiner com a imagem criada no terceiro comando, atribui este contêiner à rede *macvlan homelab*, mapeia a porta 8000 da máquina *host* à porta 8000 do contêiner, e renomeia este.

O primeiro contêiner é um servidor *echo*, que recebe uma mensagem no *payload* da requisição HTTP e retorna esse mesmo *payload*. Para interagir com tal contêiner, desenvolve-se um cliente que envia, aleatoriamente dentro de um intervalo bem definido, grandes *payloads* de texto aleatório para este servidor. A finalidade deste contêiner e seu respectivo cliente é simular eventual tráfego intensivo na rede, seu comportamento está descrito na Figura 10, a qual representa as etapas tanto do contêiner *echo* quanto da aplicação cliente, e como ocorre a comunicação entre estes.

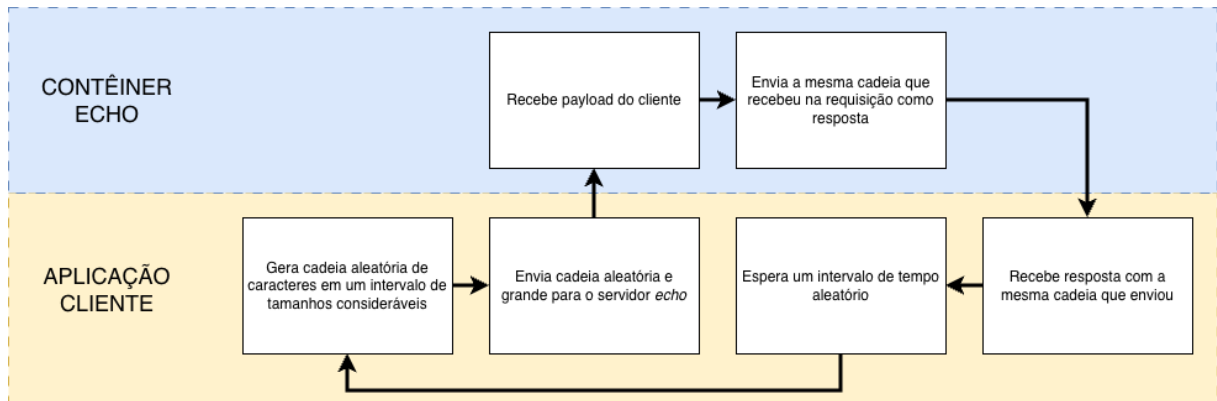


Figura 10 – Visualização do algoritmo do contêiner *echo*. Fonte: Do Autor.

O segundo contêiner desenvolvido é um *web scraper*, projetado para simular o comportamento de navegação na Internet. Este contêiner inicia sua rotina de navegar na Internet a partir de uma requisição HTTP feita por uma aplicação cliente, a qual deve informar o *website* do qual o *scraper* deve partir. Ao receber essa URL (*Uniform Resource Locator*) do cliente, o *scraper* inicia sua rotina de navegação na Internet, seguindo os passos descritos na Figura 11. Primeiro, este realiza o *download* do código-fonte HTML da URL informada pelo cliente, filtra todas as referências de *hyperlinks* na página e as envia como resposta à requisição do cliente que iniciou a comunicação. O processo cliente, por sua vez, seleciona aleatoriamente um dos *hyperlinks* retornados pelo *scraper* para visitar após um período de tempo aleatório, e faz outra requisição para o *scraper*. O *scraper* recebe a URL da página, acessa a página selecionada, realiza o *download* de seu conteúdo, filtra as referências de *hyperlinks* e as envia novamente ao cliente. Este ciclo se repete até que a aplicação cliente seja interrompida manualmente.

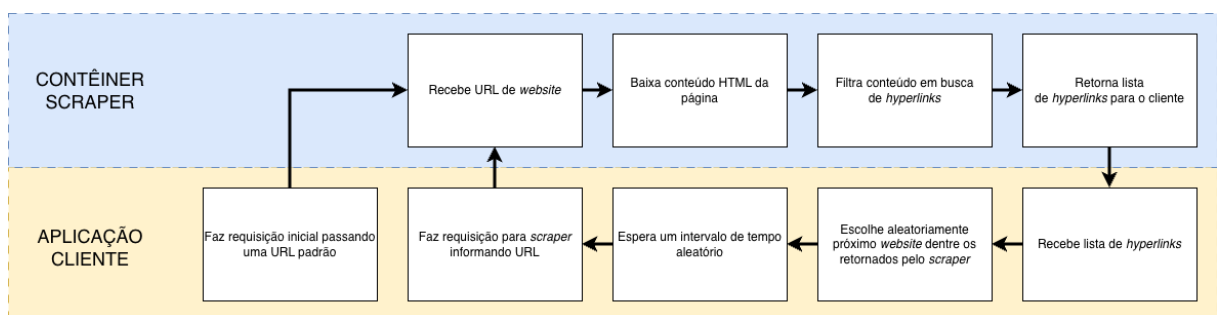


Figura 11 – Visualização do algoritmo do contêiner *scraper*. Fonte: Do Autor.

Vale ressaltar que o contêiner *scraper* e o contêiner *echo* são lançados no Algoritmo 8, com os endereços IP 192.168.1.10 e 192.168.1.11, respectivamente. Com estes dois contêineres, é possível testar não somente a comunicação entre os processos na máquina *host* e os contêineres localizados na mesma máquina, passando pelo roteador, mas também a comunicação direta dos contêineres com a Internet. Para isso, captura-se durante um dia o fluxo de pacotes da rede com esses contêineres de serviço interagindo

de maneira automática com seus respectivos processos clientes. Para isso, realiza-se a captura, durante um dia inteiro, de tráfego simulado pela comunicação dos contêineres com seus devidos clientes na máquina *host*, ao fim desta captura obtém-se um arquivo CSV com os fluxos de pacotes capturados pelo *CICFlowmeter*. Obtém-se então a Tabela 3 como subconjunto direto deste arquivo CSV gerado pela captura de tráfego simulado. Isto é feito a fim de evidenciar que a máquina *host* consegue se comunicar apropriadamente com os contêineres, tal qual ocorreria a comunicação entre diferentes dispositivos físicos na rede local.

Tabela 3 – Resultados do teste de geração de tráfego. Fonte: Do autor.

IP Origem	IP Destino	Porta Origem	Porta Destino	Pacotes Fwd	Pacotes Bwd
192.168.1.10	195.200.68.224	33640	443	53	0
192.168.1.69	52.60.82.104	33810	443	63	0
192.168.1.216	142.251.132.42	44648	443	2	0
192.168.1.216	216.239.34.223	46058	443	13	0
192.168.1.200	192.168.1.1	42225	53	2	0
192.168.1.200	192.168.1.1	58569	53	2	0
192.168.1.10	195.200.68.224	41066	443	68	0
192.168.1.69	192.168.1.200	22	56746	2	0

Na Tabela 3 é importante observar que os endereços IP 192.168.1.10 e 192.168.1.200 aparecem na captura de tráfego realizada pelo *CICFlowmeter*, evidenciando a comunicação dos contêineres com a máquina *host* e com a internet. Assim, é possível verificar que o método definido para geração de tráfego normal, utilizando apenas um computador por meio de uma rede *Docker macvlan*, contêineres disponibilizadores de serviço e *scripts* cliente, é válido para a geração de tráfego. Isso também permite concluir que a arquitetura modelada para a captura de tráfego doméstico está funcionando apropriadamente, uma vez que, pela tabela, é possível observar a comunicação de diferentes dispositivos na rede, mesmo que emulados pelos contêineres.

4.3.2 Geração de Tráfego Anômalo

Para que o *dataset* em construção possa ser utilizado no treinamento de sistemas IDS, é necessário que sua composição inclua tanto dados de tráfego normal quanto anômalo. Essa diversidade é essencial, pois os algoritmos de agrupamento ou classificação treinados com base no *dataset* devem ser capazes de distinguir entre comportamentos inofensivos e ataques na rede. Portanto, o conjunto de dados deve conter uma porção significativa de registros que representem comportamentos normais e outra porção proveniente de ataques ou simulações de ataques. Para obter os dados provenientes de ataques, não é necessário que as ofensivas sejam efetivas ou que de fato comprometam os dispositivos e serviços. Basta gerar tráfegos de pacotes que reproduzam o comportamento característico dos ataques. Dessa forma, podem ser realizadas simulações de ataques que

não causem danos reais à estrutura da rede doméstica, uma vez que, para atingir o objetivo de construir o *dataset*, é suficiente capturar uma amostra de pacotes que exibam os padrões predefinidos dos ataques.

Dito isso, esta seção aborda como são realizadas as simulações de ataques à rede. Visando priorizar a simplicidade, são escolhidos apenas dois tipos de ataques para serem aplicados de maneira controlada na rede. São eles: i) mapeamento de rede via **nmap** ([Nmap Org, 2025](#)) e ii) DoS (*Denial of Service*) via *script* com a implementação *Slow Loris*. Vale ressaltar que, mapeamento de rede não é um ataque a redes de computadores, entretanto há uma justificativa para este ter sido categorizado de tal forma, esta justificativa é abordada com detalhes na Subseção 4.3.2.1. Além disso, vale ressaltar também que os ataques são sempre aplicados aleatoriamente a um dos contêineres que foram criados na Subseção 4.3.1, e não a dispositivos reais da rede, como o dispositivo *sniffer*.

Para cada um dos ataques implementados, há um *script* distinto que os executa, todos funcionando com comportamento bem definido e semelhante. O funcionamento é descrito da seguinte forma: após executar o ataque, o *script* escolhe um tempo de espera antes de repeti-lo, sendo este tempo escolhido aleatoriamente dentro de um intervalo pré-definido. Por exemplo, após executar um mapeamento da rede com nmap, o *script* escolhe um tempo de espera aleatório no intervalo entre o tempo mínimo de cinco minutos e o tempo máximo de uma hora, dessa forma temos simulações mais dinâmicas.

Para o ataque *Slow Loris*, que pode ser executado durante um período de tempo arbitrário sem duração limitada, o *script* é executado com duração fixa de cinco minutos. Antes de apresentar o código dos *scripts*, haverá uma subseção para cada um, a fim de explicar detalhadamente seu funcionamento e compreender o impacto esperado na rede. Dessa maneira, torna-se possível identificar quando houve um ataque ao examinar o fluxo de pacotes que é capturado.

4.3.2.1 Mapeamento de Rede

Mapeamento de rede, ou *Network Mapping*, não é um tipo de ataque propriamente dito, pois pode ser utilizado para realizar o inventário de redes de computadores, organizar rotinas de atualização de serviços e dispositivos, monitorar o *uptime* de dispositivos e identificar dispositivos que mudaram de endereço IP dentro da rede. Entretanto, tal ação costuma ser incluída nessa categoria em virtude de normalmente anteceder ataques a dispositivos, uma vez que pode ser usada para encontrar dispositivos que apresentam vulnerabilidades, e também para obter uma noção de como a rede está organizada.

O escaneamento de redes pode ser implementado de diversas maneiras, mas normalmente as ferramentas que implementam tal funcionalidade realizam uma busca exploratória. Isto é, ao informar um intervalo de endereços IP de interesse, o usuário permite que as ferramentas investiguem endereço a endereço, a fim de determinar quais *hosts* es-

tão disponíveis, quais serviços estes *hosts* oferecem, quais sistemas operacionais e versões estão sendo utilizados, quais tipos de filtragem de pacotes (*firewall*) estão em uso, além de diversas outras informações, como endereços MAC e portas abertas dos dispositivos.

Escolhe-se a ferramenta NMAP ([Nmap Org, 2025](#)) para a realização das simulações de ataques no ambiente controlado estabelecido com os contêineres. A escolha de tal ferramenta se deu em razão de ser gratuita, ter uso simples via CLI, oferecer todas as funcionalidades abordadas anteriormente, ser bem documentada, oferecer amplo suporte a diferentes sistemas operacionais e funcionar em praticamente qualquer *hardware*.

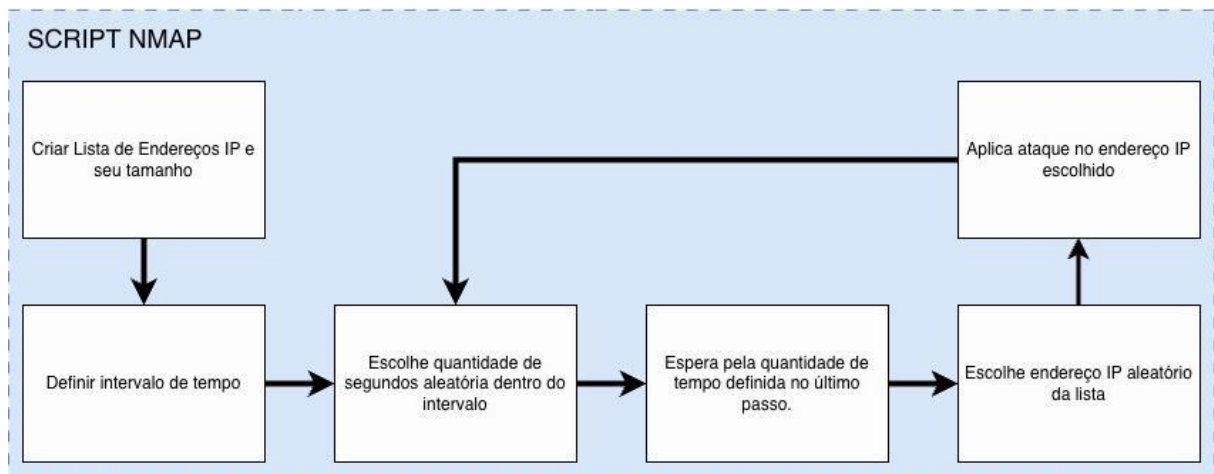


Figura 12 – Visualização do *script* de automação do ataque de mapeamento de rede.
Fonte: Do Autor.

Algorithm 9 Automação do ataque de Mapeamento de Rede. Fonte: Do autor

```
1  #!/bin/bash
2
3  # Lista com o endereço IP dos contêineres
4  declare -a TARGET_IPS=('192.168.1.1' '192.168.1.10' '192.168.1.11')
5
6  # Número de endereços IP na lista
7  NUM_IPS=${#TARGET_IPS[@]}
8
9  # 5 minutos = 300 segundos
10 # 1 hora = 3600 segundos
11 # O intervalo (range) é 3600 - 300 = 3300 segundos
12 RANGE=3301
13 MIN_TIME=300
14
15 # Loop infinito para que o script continue rodando
16 while true; do
17     # 1. Calcular o tempo de espera aleatório
18     # (dentro do intervalo [5min,1h])
19
20     # Gera um número aleatório de 0 a 3300, e então soma 300
21     # Isso garante um resultado entre 300 (5 min) e 3600 (1 hora)
22     RANDOM_TIME=$((($RANDOM % $RANGE + $MIN_TIME))
23
24     echo '[INFO] Próximo scan em $RANDOM_TIME segundos'
25     sleep $RANDOM_TIME
26
27     # 2. Escolher um IP aleatório da lista de IPs
28     RANDOM_INDEX=$((($RANDOM % $NUM_IPS))
29     TARGET_IP=${TARGET_IPS[$RANDOM_INDEX]}
30
31     # 3. Executar o Nmap Scan ###
32     echo '[INFO] $(date): Executando Nmap em $TARGET_IP...'
33     sudo nmap $TARGET_IP
34
35     echo '[INFO] Scan em $TARGET_IP concluído.'
36 done
```

O Algoritmo 9 contém o *script Bash* que descreve os ataques de mapeamento de rede aos contêineres do ambiente controlado criado anteriormente. Todo seu funcionamento é representado de maneira visual na Figura 12. É fundamental compreender o funcionamento deste *script*, uma vez que os *scripts* criados para os demais ataques seguem uma estrutura muito semelhante. Antes de iniciar o laço principal do *script*, há a criação de uma lista que contém os endereços IP dos contêineres (192.168.1.10 e 192.168.1.11) e do roteador OpenWRT (192.168.1.1) na linha 4 e há a definição do intervalo de tempo que é usado entre as execuções dos ataques nas linhas 12 e 13. Observe as linhas 28 e 29 do Algoritmo 9, note que há uma variável `$RANDOM` que não foi declarada previamente no código. Isso se deve pois, essa é uma variável global da linguagem *bash* que gera um número inteiro aleatório toda vez que é referenciada no *script*. Esta variável é usada neste e nos demais *scripts* de automação de ataques.

O laço do programa se inicia na linha 16 do Algoritmo 9 calculando um tempo de espera aleatório contido no intervalo definido anteriormente. Neste caso, o intervalo é fechado de cinco minutos a uma hora. São criadas então as variáveis `MIN_TIME` e `RANGE`, que descrevem o tempo de espera mínimo em segundos (300 segundos) e a diferença entre o intervalo completo e o tempo mínimo de espera (3301 segundos), de forma que $300 + 3301 = 3601$, valor correspondente ao intervalo completo de uma hora. Em seguida, escolhe-se um tempo aleatório dentro do intervalo completo e o programa aguarda por esta quantidade de tempo. Transcorrido este tempo, escolhe-se aleatoriamente um endereço IP da lista de endereços e é executado o mapeamento de rede.

4.3.2.2 Denial of Service – Slow Loris

Os ataques de *Denial of Service*, ou negação de serviço, consistem em ocasionar a indisponibilidade de algum serviço. Isso pode ser feito de diversas maneiras: causando *crash* da aplicação, mantendo o máximo de conexões que a aplicação suporta sempre ativas, sobrecarregando a aplicação com múltiplas requisições, entre outras formas. Em razão dessa variedade de tipos de ataque que se enquadram na categoria de *Denial of Service*, há uma ampla variedade de implementações. Portanto, o primeiro passo é escolher um tipo de ataque *Denial of Service* que possa ser aplicado, no ambiente que foi definido anteriormente com os contêineres que emulam serviços.

Como a ideia é simular ataques simples, que não exijam muito poder computacional e que sejam de fácil rastreabilidade e reprodutibilidade, ataques de *flood*, que consistem em sobrecarregar o serviço com múltiplas requisições, são inviáveis, uma vez que, em casos reais, estes são realizados de forma distribuída, utilizando múltiplos dispositivos atacantes. Assim, escolhe-se a implementação de ataque *Slow Loris*. Esta implementação é simples e consiste em abrir muitas conexões HTTP, mantendo-as abertas, sem finalizá-las por um longo período de tempo. Isso é feito mantendo a conexão aberta por meio de trocas de mensagens de maneira lenta, daí o nome do ataque *Slow Loris*. Assim, a maior parte do limite de conexões que a aplicação suporta fica sempre ocupada, tornando impossível que novos usuários façam uso do serviço, o que gera a indisponibilidade deste.

O *script* utilizado para este ataque é a implementação *open source* em Python (YALTIRAKLI, 2015), que segue a implementação clássica do ataque, sem otimizações e variações em seu funcionamento. Essa implementação foi escolhida em razão de sua simplicidade de uso, por ser código aberto e por ser uma implementação básica que segue rigorosamente a teoria. Para a instalação do *script* é seguido o passo-a-passo de instalação definido no repositório oficial da ferramenta.

O Algoritmo 10 contém o *script Bash* que descreve os ataques *Slow Loris* aos contêineres do ambiente controlado criado anteriormente. O esboço do seu funcionamento é representado de maneira visual na Figura 13.

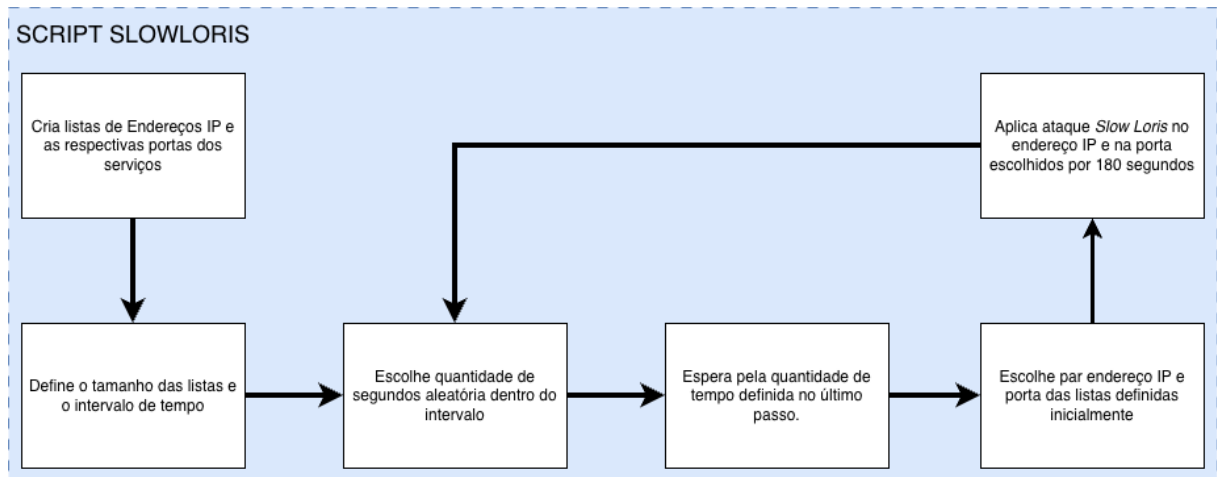


Figura 13 – Visualização do *script* de automação do ataque *Slow Loris*. Fonte: Do Autor.

O fluxo deste *script* (Algoritmo 10) é semelhante ao do *script* apresentado na Seção 4.3.2.1. Face ao exposto, o fluxo é comentado de maneira menos detalhada. Antes de iniciar o laço principal do *script*, há a criação de uma lista que contém os endereços IP dos contêineres (192.168.1.10 e 192.168.1.11) na linha 4 e uma lista para suas respectivas portas na linha 5, além da definição do intervalo de tempo que é utilizado entre as execuções dos ataques nas linhas 13 e 14.

Algorithm 10 Automação do ataque *Slow Loris*. Fonte: Do autor

```

1  #!/bin/bash
2
3  # Listas com o endereço IP dos contêineres e suas respectivas portas abertas
4  declare -a TARGET_IPS=('192.168.1.10' '192.168.1.11')
5  declare -a TARGET_PORTS=('8000' '8001')
6
7  # Número de endereços IP na lista
8  NUM_TARGETS=${#TARGET_IPS[@]}
9
10 # 5 minutos = 300 segundos
11 # 1 hora = 3600 segundos
12 # O intervalo (range) é 3600 - 300 = 3300
13 RANGE=3301
14 MIN_TIME=300
15
16 while true; do
17     # 1. Calcular o tempo de espera aleatório
18     # (dentro do intervalo [5min,1h])
19
20     # Gera um número aleatório de 0 a 3300, e então soma 300
21     # Isso garante um resultado entre 300 (5 min) e 3600 (1 hora)
22
23     RANDOM_TIME=$((($RANDOM % $RANGE + $MIN_TIME))
24
25     echo '[INFO] Próximo teste em $RANDOM_TIME segundos'
26     sleep $RANDOM_TIME
27
28     # 2. Escolher um par de IP/Porta aleatório
29     RANDOM_INDEX=$((($RANDOM % $NUM_TARGETS))
30     TARGET_IP=${TARGET_IPS[$RANDOM_INDEX]}
31     TARGET_PORT=${TARGET_PORTS[$RANDOM_INDEX]}
32
33     # 3. Executar o slowloris por 180s (3 minutos) no alvo
34     timeout 180s slowloris -p '$TARGET_PORT' '$TARGET_IP'
35
36     echo '[INFO] Teste em $TARGET_IP:$TARGET_PORT concluído.'
37 done

```

O laço do programa se inicia na linha 16, calculando um tempo de espera aleatório contido no intervalo definido anteriormente. Neste caso, o intervalo é fechado de cinco minutos a uma hora. Em seguida, escolhe-se um tempo aleatório dentro do intervalo completo e o programa aguarda por esta quantidade de tempo. Transcorrido este tempo, escolhe-se aleatoriamente um par endereço IP e porta das listas de endereços e portas. Então, o ataque *Slow Loris* é executado por um período de 180 segundos (3 minutos), utilizando o comando `timeout` para limitar a duração do ataque.

4.3.2.3 Ambiente de Execução e Identificação dos Ataques

Visando isolar o ambiente de ataques e simplificar o processo de rotulagem dos dados, foi criado um contêiner *Docker* dedicado exclusivamente à execução dos *scripts* de

automação dos ataques Mapeamento de Rede e *Slow Loris*. A especificação técnica deste ambiente é apresentada no Algoritmo 11.

Algorithm 11 *Dockerfile* do contêiner que executa os ataques. Fonte: Do autor

```
1 FROM python:3-slim
2
3 # 1. instala-se as dependências de sistema (Nmap e Git)
4 RUN apt-get update && \
5     apt-get install -y nmap git procps && \
6     rm -rf /var/lib/apt/lists/*
7
8 # 2. instala-se o slowloris do repositório gkbrk
9 # o repositório é clonado
10 # então este torna-se executável
11 # e por fim, é criado um link simbólico para o PATH do executável
12 RUN git clone https://github.com/gkbrk/slowloris.git /opt/slowloris && \
13     chmod +x /opt/slowloris/slowloris.py && \
14     ln -s /opt/slowloris/slowloris.py /usr/local/bin/slowloris
15
16 # 3. configura-se diretório de trabalho
17 WORKDIR /app
18
19 # 4. copia-se os scripts para dentro do container
20 COPY scanner.sh /app/scanner.sh
21 COPY attacker.sh /app/attacker.sh
22 COPY start.sh /app/start.sh
23
24 # 5. é garantida permissão de execução aos scripts
25 RUN chmod +x /app/scanner.sh /app/attacker.sh /app/start.sh
26
27 # 6. define-se o entrypoint
28 CMD ["/start.sh"]
```

O processo de construção da imagem, detalhado no Algoritmo 11, inicia-se na primeira linha com a definição de uma imagem base *Python* em sua versão *slim*, escolhida por ser leve e conter o interpretador necessário para a ferramenta *Slow Loris* (YALTIRAKLI, 2015). As linhas 4 a 6 encarregam-se de atualizar os repositórios e instalar dependências essenciais: o *Nmap* para o mapeamento de rede, o *Git* para clonagem de repositórios e o pacote *procps* para gestão de processos. Entre as linhas 12 e 14, realiza-se a clonagem do repositório oficial do *Slow Loris*, concedendo-se permissão de execução ao *script* principal e criando um *link* simbólico para facilitar sua chamada via linha de comando. Na sequência, define-se o diretório de trabalho (linha 17) e copiam-se os *scripts* de automação desenvolvidos (*scanner.sh*, *attacker.sh* e o orquestrador *start.sh*) para o interior do contêiner, garantindo-lhes permissão de execução (linha 25). Por fim, a linha 28 define o *entrypoint*, que instrui o contêiner a iniciar imediatamente a execução simultânea das rotinas de ataque ao ser carregado.

Instancia-se então o contêiner na mesma rede *macvlan* dos serviços alvo, conforme

demonstrado no Algoritmo 12.

Algorithm 12 Construção da imagem e execução do contêiner de ataques. Fonte: Do autor

```
1 # construcao da imagem anomaly
2 sudo docker build -t anomaly anomaly
3
4 # execucao do container anomaly com IP estático
5 sudo docker run -d \
6     --network homelab \
7     --ip 192.168.1.13 \
8     --name anomaly \
9     anomaly
```

O comando de execução define explicitamente que o contêiner deve operar na rede *homelab* e atribui a ele o endereço IP estático 192.168.1.13. A atribuição de um endereço IP exclusivo e fixo para a origem do tráfego anômalo oferece uma vantagem significativa para a construção do *dataset*: a rotulação determinística. Durante a fase de processamento dos dados e geração do *Ground Truth*, qualquer fluxo de pacotes que tenha como origem o endereço 192.168.1.13 é automaticamente classificado como "ataque", enquanto o tráfego proveniente de quaisquer outros endereços é considerado benigno (normal). Essa abordagem elimina ambiguidades na classificação supervisionada e assegura a integridade dos rótulos utilizados no treinamento do IDS.

4.3.2.4 Validação Preliminar da Geração de Tráfego Anômalo

Após a implementação dos dois tipos de simulações de ataques descritos nas subseções anteriores: i) mapeamento de rede via *NMAP* (Nmap Org, 2025) e ii) *Denial of Service* com *Slow Loris* (YALTIRAKLI, 2015), torna-se necessário validar o funcionamento do ambiente proposto antes de prosseguir para a Seção 4.4 à qual é dedicada aos experimentos propriamente ditos. Esta validação preliminar tem como objetivo comprovar que os fluxos de pacotes provenientes das simulações de ataques são capturados pelo *sniffer* e adequadamente processados pelo *CICFlowmeter*, demonstrando que o ambiente criado para a geração de tráfego anômalo está operacional e produzindo *datasets* válidos.

Para realizar esta validação, executa-se um experimento controlado com duração de uma hora e meia. Durante este período, todos os contêineres de serviço permanecem ativos, mas sem seus respectivos *scripts* clientes, a fim de facilitar a identificação manual dos fluxos de pacotes maliciosos. Simultaneamente, os dois *scripts* de automação de ataques: mapeamento de rede e *Slow Loris*, são executados a partir do contêiner atacante, aplicando seus respectivos ataques de forma aleatória aos contêineres alvo, seguindo os intervalos e durações estabelecidos em suas implementações. O tráfego de rede resultante desta combinação de atividades normais e anômalas é capturado e processado continuamente

pelo *CICFlowmeter* para a extração das características dos fluxos de pacotes e construção de um pequeno *dataset*.

Tabela 4 – Resultados do teste de geração de tráfego anômalo. Fonte: Do autor.

Linha	IP Origem	IP Destino	Porta Origem	Porta Destino	Pacotes Fwd	Pacotes Bwd
1	192.168.1.69	142.251.132.42	44648	443	45	0
2	192.168.1.13	192.168.1.1	42225	53	2	0
3	192.168.1.13	192.168.1.10	54321	8000	1254	0
4	192.168.1.216	195.200.68.224	33640	443	53	0
5	192.168.1.13	192.168.1.11	45678	0	156	0
6	192.168.1.216	216.239.34.223	46058	443	13	0
8	192.168.1.69	52.60.82.104	33810	443	63	0
9	192.168.1.13	192.168.1.11	48923	8001	1189	0
10	192.168.1.216	195.200.68.224	41066	443	68	0
11	192.168.1.13	192.168.1.1	52341	0	124	0
12	192.168.1.216	142.251.132.42	51234	443	28	0

Na Tabela 4 é possível observar a presença de tráfego anômalo proveniente das simulações de ataques. As linhas 3 e 9 evidenciam ataques *Slow Loris* originados do contêiner atacante, identificado pelo endereço IP 192.168.1.13. Estes ataques são direcionados aos contêineres de serviço 192.168.1.10 e 192.168.1.11 nas portas 8000 e 8001, respectivamente. Estes fluxos são caracterizados pelo elevado número de pacotes enviados sem resposta, isto é, pacotes *Fwd* com valores elevados, padrão típico deste tipo de ataque. As linhas 5 e 11 demonstram ataques de mapeamento de rede ao roteador OpenWrt, também originados em 192.168.1.13, evidenciados pelo acesso à porta 0 ou varredura de múltiplas portas. As demais linhas representam tráfego normal gerado pela comunicação regular dos outros dispositivos presentes na rede, como celulares e computadores pessoais (exemplo: linhas com origem 192.168.1.216 e 192.168.1.69).

Dessa forma, verifica-se que o método definido para geração de tráfego anômalo, utilizando um contêiner dedicado para execução dos *scripts* de automação de ataques, é válido e eficiente para a construção do *dataset*. Os resultados demonstram que a arquitetura proposta opera adequadamente, uma vez que os fluxos de pacotes provenientes das simulações de ataques são facilmente identificáveis pelo IP de origem, capturados pelo *sniffer* e processados corretamente pelo *CICFlowmeter*. Isso permite concluir que o ambiente está preparado para a execução dos experimentos extensivos.

4.4 Experimentos

Uma vez validada a arquitetura de rede proposta e verificada a eficácia dos *scripts* de geração de tráfego normal e anômalo, torna-se possível avançar para a etapa de coleta massiva de dados. O objetivo central desta seção é demonstrar a viabilidade do modelo desenvolvido para a criação de *datasets* voltados ao treinamento de sistemas de Detecção de Intrusão (IDS), submetendo o ambiente a cenários de uso controlado e misto. A re-

alização destes experimentos visa não apenas a construção dos conjuntos de dados, mas também a avaliação do comportamento do *sniffer* e do fluxo de extração de características pelo *CICFlowmeter* sob diferentes cargas de trabalho e durações.

Para garantir a consistência e a robustez dos dados gerados, os experimentos foram divididos em duas categorias, detalhadas nas subseções seguintes: experimentos em ambiente totalmente simulado e experimentos em ambiente real. Em ambas as categorias, adota-se uma metodologia progressiva de duração dos testes, a fim de validar a estabilidade da coleta de dados. São propostos dois cenários de captura para cada ambiente: um teste de média duração (12 horas) e um teste de longa duração (24 horas). Essa variação temporal é fundamental para observar se padrões de tráfego, tanto normais quanto anômalos, mantêm-se identificáveis ao longo de janelas de tempo estendidas e se o volume de dados gerado é gerenciável.

Ao final de cada ciclo de testes, os arquivos gerados pelo *CICFlowmeter* são analisados quanto a integridade dos fluxos capturados. Espera-se que os resultados evidenciem a capacidade da arquitetura em segregar corretamente os fluxos de tráfego benigno daqueles provenientes das simulações de ataques, consolidando assim a utilidade da ferramenta para a pesquisa em segurança de redes e aprendizado de máquina.

4.4.1 Experimentos com Tráfego Simulado

Nesta etapa de validação, o ambiente de testes é configurado de maneira estritamente controlada e isolada, utilizando exclusivamente a infraestrutura de contêineres *Docker* descrita na Seção 4.3. O cenário é composto pelos contêineres prestadores de serviço (servidor *echo* e *web scraper*), que interagem com seus respectivos *scripts* clientes executados na máquina *host*, simulando o comportamento de usuários legítimos. Simultaneamente, os *scripts* de automação de ataques (Mapeamento de rede e *Slow Loris*) atuam sobre esses serviços de forma aleatória. O objetivo deste isolamento é estabelecer um "Ground Truth" preciso, em que todas as variáveis de interação na rede são conhecidas e geradas de forma simulada, eliminando ruídos externos imprevisíveis.

A metodologia para esta etapa consiste na execução de duas baterias de testes. O primeiro teste, com duração de doze horas, visa simular uma jornada de trabalho ou uso diurno intenso, avaliando a consistência dos *scripts* de automação por um período significativo. O segundo teste busca validar a estabilidade da arquitetura em um ciclo diário completo de 24 horas, garantindo que não ocorram falhas de segmentação ou estouro de recursos durante capturas longas. Durante os testes, o tráfego capturado e processado em tempo real é armazenado em arquivos CSV que representam o comportamento dos serviços implementados.

A análise dos dados resultantes destes experimentos permite confirmar se os pa-

drões de tráfego normal simulados pelos *scripts* clientes assemelham-se estatisticamente a comportamentos aceitáveis e se os ataques destacam-se adequadamente no conjunto de dados. Como o tráfego é 100% sintético durante a bateria de testes, qualquer anomalia não classificada ou comportamento inesperado no *dataset* final indica a necessidade de ajustes nos parâmetros de geração de tráfego, na configuração do *sniffer*, ou até mesmo na topologia da rede. Esta etapa é crucial para garantir que a "sintaxe" dos ataques e do tráfego normal esteja correta antes da introdução de variáveis do mundo real.

4.4.1.1 Experimento de Média Duração (12 Horas)

O experimento de média duração foi conduzido com o objetivo primário de avaliar a estabilidade da coleta de dados e a consistência dos *scripts* de geração de tráfego em um período estendido, simulando uma jornada de trabalho ou um ciclo de atividade diurna. O teste teve duração exata de 12 horas, durante as quais o ambiente permaneceu isolado e monitorado, contando somente com os contêineres de serviço, contêiner executor dos ataques e *scripts* clientes dos serviços sendo executados em um computador pessoal.

A Tabela 5 apresenta o resumo estatístico dos dados coletados. Observa-se a captura de um total de 24.452 fluxos de rede. Um ponto crucial para a viabilidade do *dataset* em aplicações de aprendizado de máquina é o balanceamento entre as classes. Neste cenário, obteve-se uma distribuição de 68,6% de fluxos normais e 31,4% de fluxos anômalos.

Tabela 5 – Resumo geral das métricas do experimento com tráfego simulado de média duração. Fonte: Do autor.

Métrica	Valor
Duração Total (h)	12.00
Total de Fluxos	24452
Fluxos Normais	16774 (68.6%)
Fluxos Anômalos	7678 (31.4%)
Protocolos Distintos	2
IPs Únicos (Origem)	9
IPs Únicos (Destino)	216

A proporção visual entre tráfego normal e anômalo pode ser observada na Figura 14. Esta distribuição indica que os *scripts* de ataque (*Slow Loris* e mapeamento de rede) atuaram com frequência suficiente para criar uma massa de dados significativa, sem, no entanto, sobrepujar completamente o tráfego legítimo, o que poderia enviesar o treinamento de uma máquina de aprendizado para a detecção de ataques.

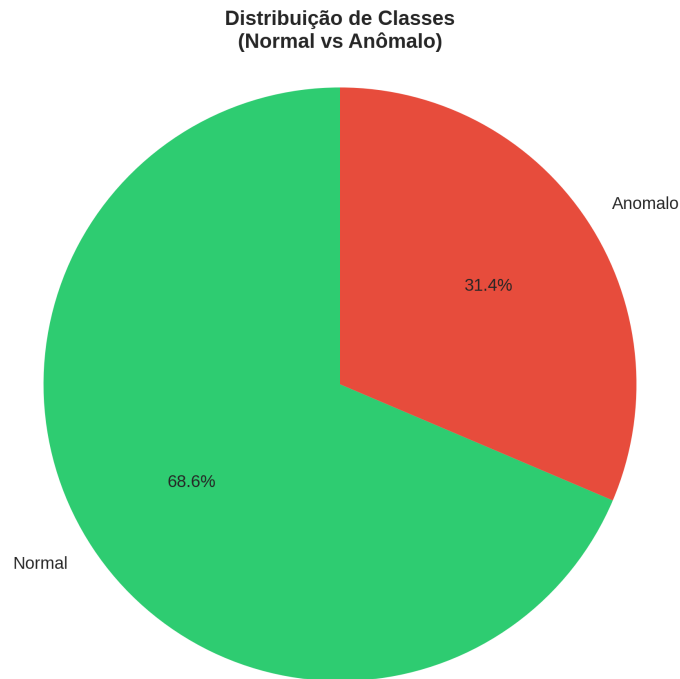


Figura 14 – Distribuição de classes no experimento com tráfego simulado de média duração. Fonte: Do autor.

Para compreender a dinâmica temporal do tráfego, a Figura 15 apresenta a série temporal do volume de fluxos a cada 5 minutos. O gráfico demonstra a natureza "pulsante" e variável do tráfego gerado. Nota-se que não houve interrupções na captura (*gaps*), o que valida a robustez da arquitetura com os contêineres *Docker* proposta. Os picos observados correspondem aos momentos de atuação simultânea dos serviços de *web scraping*, e dos *scripts* de ataque, enquanto os vales representam os intervalos de latência programados nos clientes para simular o comportamento humano. Vale ressaltar que a presença de um *plateau* é justificada pelo tráfego gerado passivamente pelos dispositivos conectados à rede.

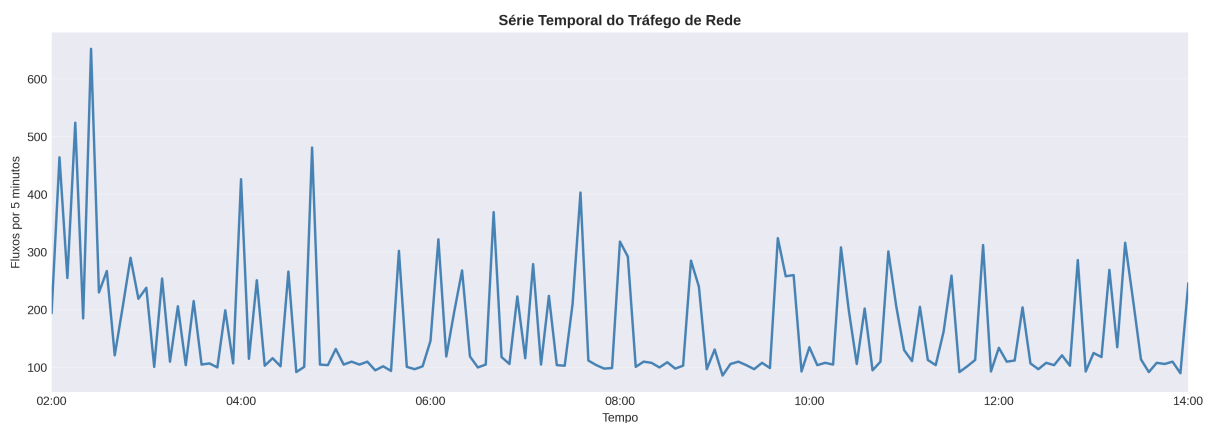


Figura 15 – Série temporal do experimento com tráfego simulado de média duração. Fonte: Do autor.

Uma análise das portas de destino, apresentada na Tabela 6 e visualizada na Figura 16, revela o perfil dos serviços acessados. O domínio da porta 53 (DNS), com 34,82% dos fluxos, seguida pela porta 443 (HTTPS) com 26,36%, é compatível com o comportamento esperado do serviço *web scraper*, que realiza constantes resoluções de nomes de domínio e requisições HTTPS para o *download* do conteúdo das páginas *web*.

Tabela 6 – Top 10 portas de destino mais acessadas no experimento com tráfego simulado de média duração. Fonte: Do autor.

Porta Destino	Quantidade de Fluxos	% do Total
53	8514	34.82
443	6446	26.36
3478	750	3.07
80	447	1.83
50568	349	1.43
1900	196	0.80
5351	190	0.78
22	106	0.43
52384	105	0.43
23	52	0.21

A presença de portas com altos valores numéricos (como 50568 e 52384) e portas de serviços específicos (como 23 para *Telnet*, alvo comum de ataques de força bruta e mapeamento de rede) confirma que o ambiente, mesmo sendo simulado, conseguiu gerar uma diversidade ampla de conexões em diferentes portas.

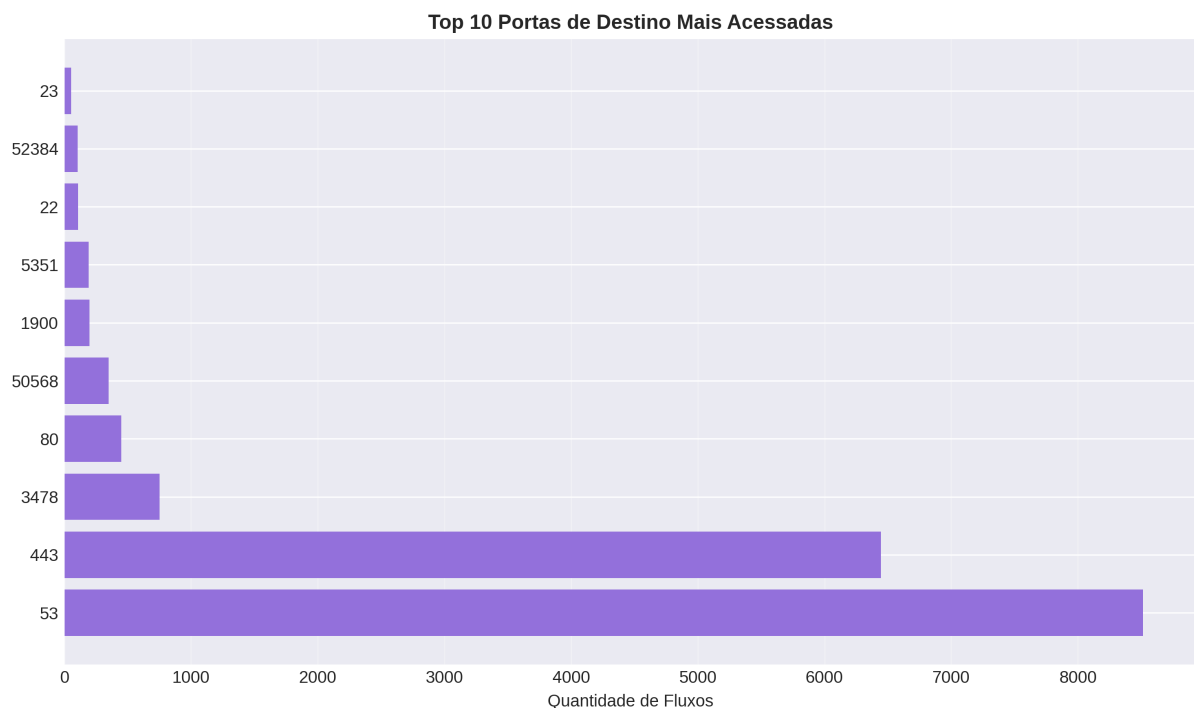


Figura 16 – Visualização das 10 portas de destino com maior volume de tráfego no experimento com tráfego simulado de duração média. Fonte: Do autor.

Por fim, a análise de correlação entre as *features* extraídas pelo *CICFlowmeter* é apresentada no *heatmap* da Figura 17. Esta análise é fundamental para verificar a redundância e a relevância dos atributos. Observa-se uma forte correlação positiva (próxima de 1.0) entre o número total de pacotes (*tot_fwd_pkts*, *tot_bwd_pkts*) e o comprimento total dos dados (*totlen_fwd_pkts*, *totlen_bwd_pkts*), o que é de fato esperado.

Entretanto, métricas como a duração do fluxo (*flow_duration*) apresentam baixa correlação com as taxas de *bytes* por segundo (*flow_bytes_s*), indicando que estas características trazem informações distintas e complementares para o modelo de classificação. Isso sugere que o conjunto de dados possui riqueza estatística suficiente para diferenciar comportamentos complexos.

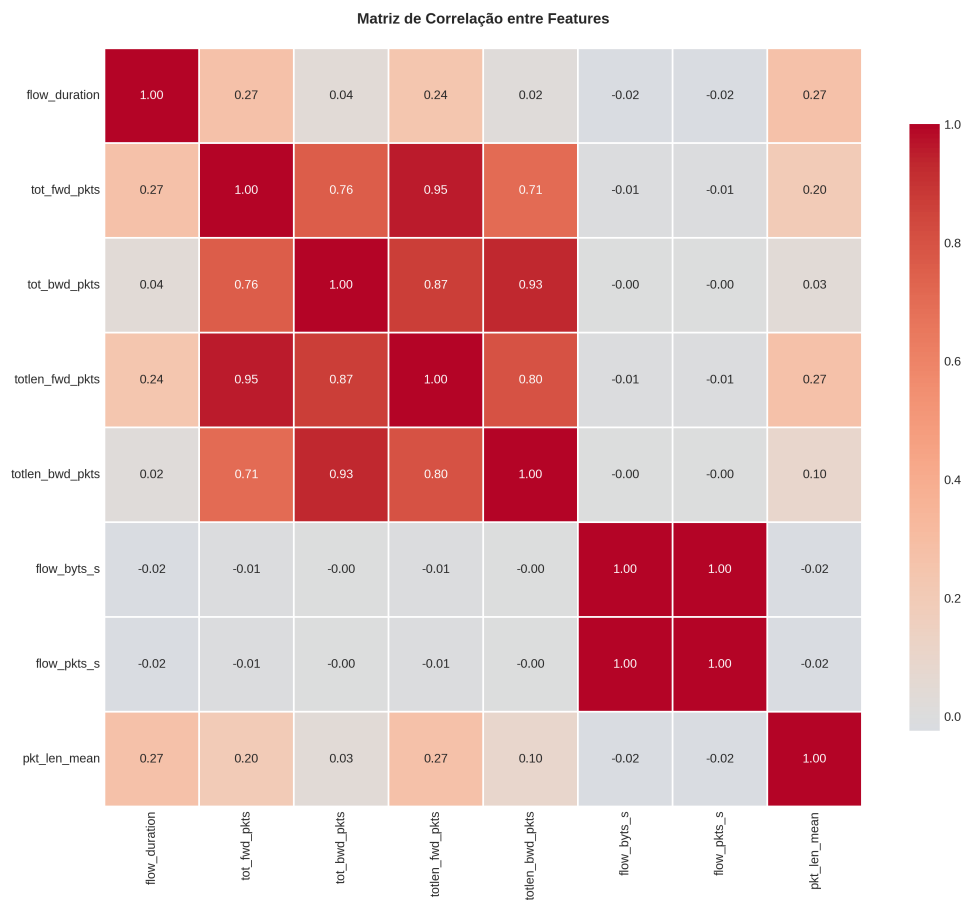


Figura 17 – *Heatmap* de correlação entre as principais *features* extraídas no experimento com tráfego simulado de média duração. Fonte: Do autor.

Os resultados obtidos neste experimento de 12 horas validam a capacidade do ambiente de gerar e capturar tráfego de forma contínua e estável, fornecendo a segurança necessária para a execução do experimento de longa duração (24 horas).

4.4.1.2 Experimento de Longa Duração (24 Horas)

O experimento de longa duração, realizado ao longo do período de 24 horas, representa a validação final da arquitetura em ambiente estritamente controlado. O objetivo principal desta etapa é verificar a estabilidade do *sniffer* e a integridade dos arquivos gerados sob uso contínuo, assegurando que não ocorressem falhas de segmentação de memória, estouro de *buffer* ou interrupções na orquestração dos contêineres *Docker*.

A Tabela 7 sumariza as métricas obtidas. O volume total de 25.249 fluxos demonstra a capacidade da ferramenta em lidar com uma quantidade massiva de dados. Diferentemente do experimento de 12 horas, observou-se aqui uma leve inversão na distribuição das classes, com o tráfego anômalo representando 54,3% do total e o tráfego normal 45,7%. Essa variação ocorreu em razão de, no longo prazo, os ataques automatizados gerarem mais fluxos de pacotes que os *scripts* clientes dos contêineres de serviço.

Tabela 7 – Resumo geral das métricas do experimento com tráfego simulado de longa duração. Fonte: Do autor.

Métrica	Valor
Duração Total (h)	24.00
Total de Fluxos	25249
Fluxos Normais	11546 (45.7%)
Fluxos Anômalos	13703 (54.3%)
Protocolos Distintos	2
IPs Únicos (Origem)	13
IPs Únicos (Destino)	410

A distribuição das classes está disponível para visualização na Figura 18. O equilíbrio próximo de 50/50 entre as classes confirma que os *scripts* de ataque mantiveram sua eficácia ao longo de todo o dia, e que o tráfego benigno foi gerado de forma consistente, simulando um ambiente de rede sob estresse e uso constante durante todo o dia.

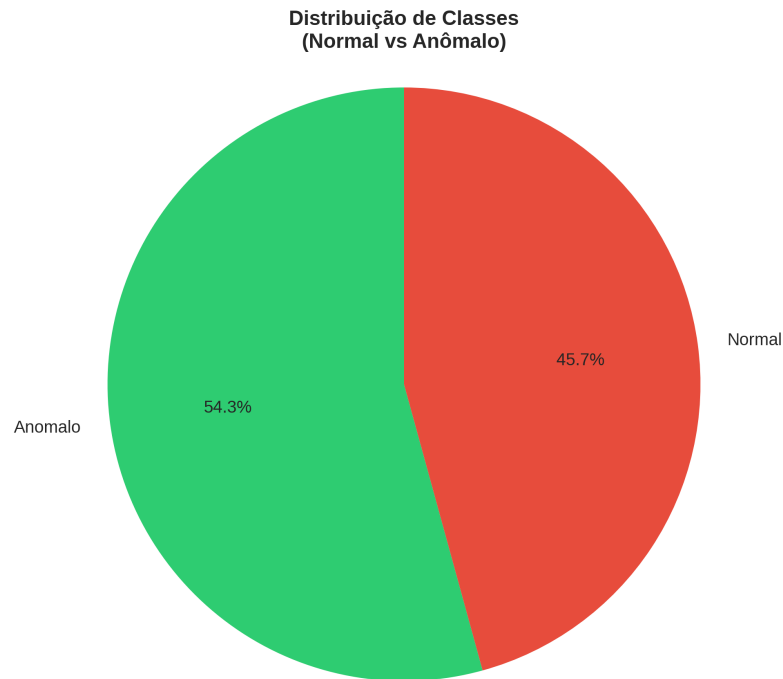


Figura 18 – Distribuição de classes no experimento com tráfego simulado de longa duração. Fonte: Do autor.

A estabilidade temporal da captura é evidenciada na Figura 19. O gráfico abrange o período completo de um dia (indicado pelo eixo X de 12 de Dezembro), exibindo a variação do tráfego sem apresentar lacunas (*gaps*) de dados. Os picos de tráfego, que chegam a ultrapassar 400 fluxos por intervalo de 5 minutos, demonstram momentos de alta atividade em que as requisições dos contêineres de serviço coincidiram com as execuções dos ataques automatizados. Isto permite a validação da alta capacidade de processamento da infraestrutura proposta para captura de fluxos de pacotes.

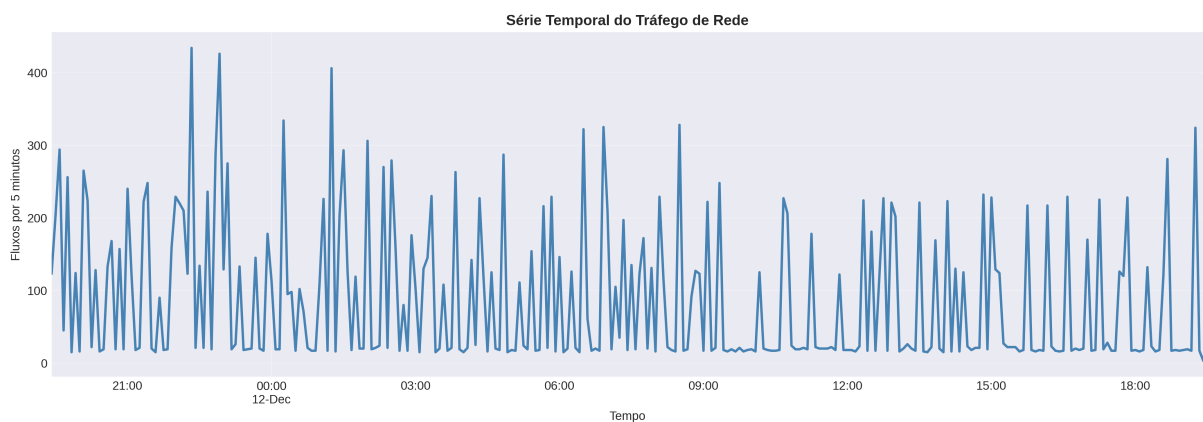


Figura 19 – Série temporal do experimento com tráfego simulado de longa duração. Fonte: Do autor.

Ao analisar as portas de destino mais acessadas na Tabela 8 e na Figura 20, nota-se

uma mudança no perfil em relação ao experimento de 12 horas. Neste cenário de longa duração, a porta 443 (HTTPS) assumiu a liderança com 22,74% dos fluxos, seguida pela porta 53 (DNS) com 16,64%. Isso indica um volume maior de transferência de dados criptografados em relação às resoluções de nomes, consistente com sessões de navegação e *scraping* mais longas e profundas.

Tabela 8 – Top 10 portas de destino mais acessadas no experimento com tráfego simulado de longa duração. Fonte: Do autor.

Porta Destino	Quantidade de Fluxos	% do Total
443	5741	22.74
53	4202	16.64
80	716	2.84
56230	701	2.78
44296	225	0.89
22	166	0.66
52384	143	0.57
123	87	0.34
995	82	0.32
139	82	0.32

A aparição de portas como 123 (NTP) e 139 (*NetBIOS*) sugere a captura de tráfego de infraestrutura e manutenção da rede, além das conexões geradas explicitamente pelos *scripts*. Além disso, evidencia-se que, além do tráfego gerado de forma simulada, há também tráfego passivo dos dispositivos conectados à rede sendo capturado. A presença de portas altas (ex: 56230) continua indicando o uso de portas efêmeras para conexões de saída ou tentativas de varredura aleatória pelo *script* de mapeamento de rede.

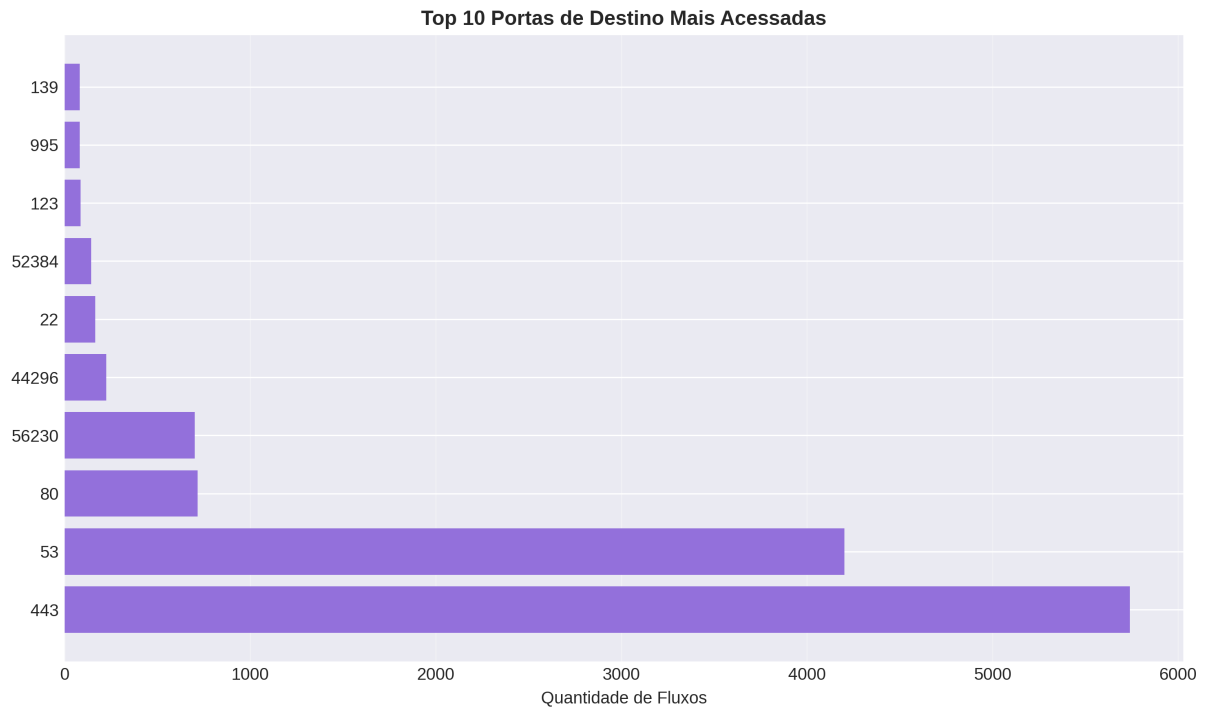


Figura 20 – Visualização das 10 portas de destino com maior volume de tráfego no experimento de 24 horas. Fonte: Do autor.

Finalmente, a matriz de correlação apresentada na Figura 21 reafirma a consistência das *features* extraídas. Assim como nos experimentos anteriores, mantém-se a coerência lógica em que variáveis de contagem de pacotes correlacionam-se fortemente com o volume de *bytes*, enquanto variáveis de tempo e taxa permanecem estatisticamente independentes. Isso confirma que o aumento da duração do experimento não introduziu ruídos ou anomalias estatísticas na formação dos fluxos pelo *CICFlowmeter*.

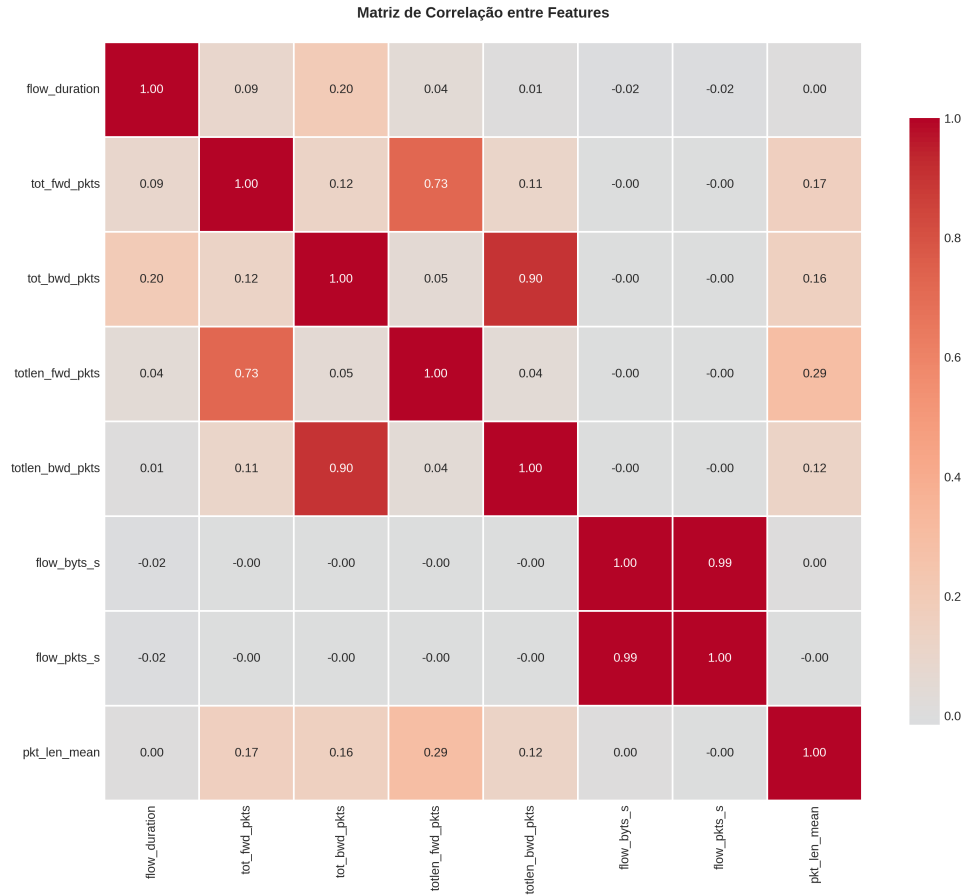


Figura 21 – *Heatmap* de correlação entre as principais *features* extraídas no experimento de 24 horas. Fonte: Do autor.

A conclusão bem-sucedida do experimento de 24 horas atesta a robustez do ambiente de simulação. O conjunto de dados gerado apresenta diversidade de protocolos, balanceamento de classes e consistência temporal, estando apto para ser utilizado no treinamento e validação de sistemas de detecção de intrusão, cumprindo assim o objetivo de criar um *dataset* confiável em ambiente controlado.

4.4.2 Experimentos em Ambiente Real

Após a validação do modelo em ambiente controlado, os experimentos avançam para um cenário híbrido e mais complexo, denominado ambiente real. Neste contexto, os *scripts* clientes que simulam o tráfego normal são desativados. Em seu lugar, a geração de tráfego benigno passa a ser responsabilidade dos dispositivos físicos reais conectados à rede doméstica, como *smartphones*, *laptops*, dispositivos IoT e televisores inteligentes, utilizados de maneira orgânica pelos residentes. A infraestrutura de ataques e os contêineres de serviço alvo permanecem ativos, injetando tráfego malicioso de forma controlada em meio ao fluxo de dados real da residência.

Assim como na etapa anterior, a metodologia de validação segue a progressão tem-

poral de testes de 12 e 24 horas. A introdução de dispositivos reais adiciona uma camada significativa de entropia e imprevisibilidade ao *dataset*, uma vez que o comportamento humano e as comunicações automáticas dos dispositivos conectados à rede não seguem padrões rígidos como os *scripts* geradores de tráfego normal. O teste de 24 horas, neste cenário, é particularmente relevante, pois captura as variações naturais de uso da rede ao longo do dia e da noite, contrastando períodos de ocio com momentos de alto consumo de banda, enquanto os ataques ocorrem periodicamente em segundo plano, direcionados aos contêineres isolados na rede *Docker macvlan*.

Estes experimentos são realizados a fim de obter um *dataset* que mescla a complexidade do tráfego real com ataques rotulados e controlados. A análise dos arquivos gerados pelo *CICFlowmeter* neste cenário foca na capacidade de distinção dos fluxos. É fundamental verificar se o tráfego gerado pelos ataques simulados não é mascarado pelo ruído do tráfego doméstico legítimo e se a arquitetura proposta é capaz de capturar com fidelidade as nuances de uma rede viva, mantendo a precisão dos rótulos (ataque *versus* normal) necessária para o treinamento eficaz de algoritmos de aprendizado de máquina.

4.4.2.1 Experimento de Média Duração (12 Horas)

O experimento de média duração em ambiente real tem como foco avaliar o comportamento da arquitetura híbrida durante um período de atividade intensa, correspondente a uma meia jornada diária. Nesta etapa, a geração de tráfego benigno ocorre de maneira orgânica por meio dos dispositivos dos usuários da rede, enquanto os ataques são injetados por meio do *script* de automação dos ataques.

A Tabela 9 apresenta o sumário estatístico dos dados coletados. O volume total de 31.258 fluxos em apenas 12 horas evidencia a densidade de tráfego de uma rede doméstica em uso ativo. Nota-se um desbalanceamento de classes mais acentuado do que nos cenários artificiais, com 79,1% de fluxos normais e 20,9% de fluxos anômalos. Este comportamento reflete a realidade de ambientes domésticos, em que o tráfego legítimo supera largamente as ocorrências de intrusão.

Tabela 9 – Resumo geral das métricas do experimento em ambiente real de média duração.
Fonte: Do autor.

Métrica	Valor
Duração Total (h)	12.00
Total de Fluxos	31258
Fluxos Normais	24719 (79.1%)
Fluxos Anômalos	6539 (20.9%)
Protocolos Distintos	2
IPs Únicos (Origem)	19
IPs Únicos (Destino)	1045

A predominância do tráfego benigno é ilustrada na Figura 22. A ampla fatia cor-

respondente à classe "Normal" valida a hipótese de que a introdução de dispositivos reais (como *smartphones* e *laptops*) enriquece significativamente o conjunto de dados, diluindo a presença dos ataques simulados sem, contudo, torná-los invisíveis.

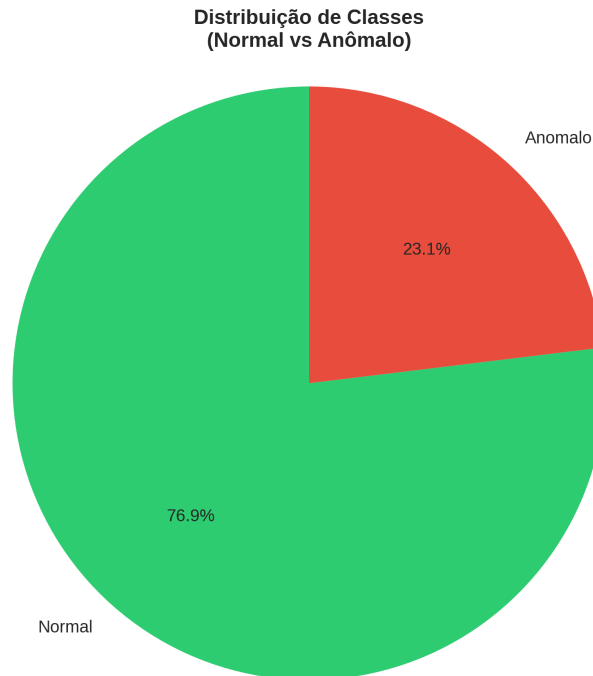


Figura 22 – Distribuição de classes no experimento em ambiente real de média duração. Fonte: Do autor.

A série temporal, apresentada na Figura 23, demonstra a variabilidade estocástica típica do uso humano. Este gráfico exibe flutuações irregulares e picos de tráfego que correspondem a momentos de interação direta dos usuários com a rede, intercalados com o tráfego gerado pelos serviços de ataque e tráfego passivo dos dispositivos conectados à rede. A ausência de lacunas na captura confirma a estabilidade da arquitetura proposta mesmo sob esta carga dinâmica e com fluxo intenso.

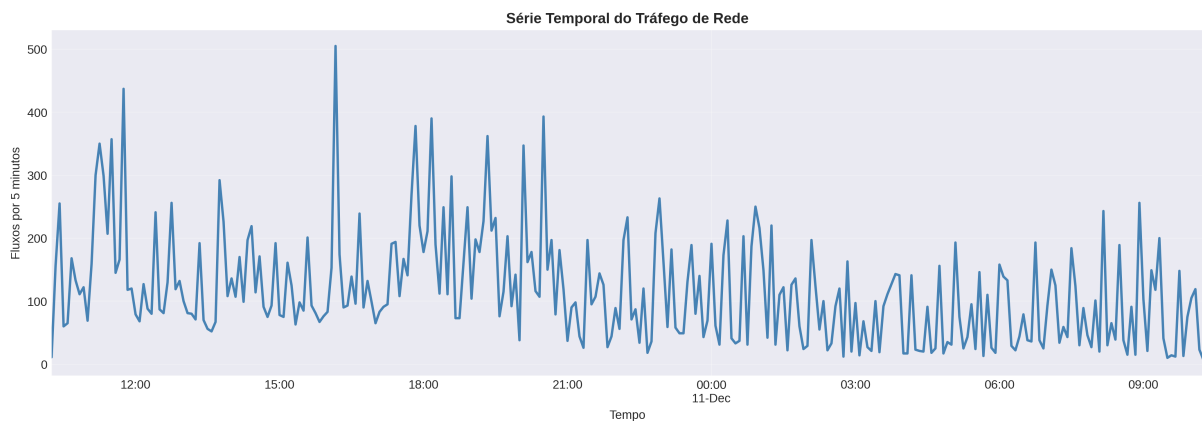


Figura 23 – Série temporal do experimento em ambiente real de média duração. Fonte: Do autor.

O perfil das conexões é detalhado na análise das portas de destino na Tabela 10 e na Figura 24. A porta 443 (HTTPS) lidera com 55,32% dos fluxos, seguida pela porta 53 (DNS) com 18,21%, o que é consistente com a navegação segura moderna. Destaca-se, neste cenário de 12 horas, a presença marcante de serviços móveis e de descoberta de rede, ausentes nos testes puramente artificiais. As portas 5228 (*Google Services*) e 5222 (XMPP/*WhatsApp*) evidenciam a atividade de dispositivos *Android* na rede. Simultaneamente, as portas 5353 (mDNS) e 1900 (SSDP) indicam a comunicação contínua entre dispositivos inteligentes (IoT) para descoberta de serviços locais, adicionando uma camada de complexidade e realismo fundamental ao *dataset*.

Tabela 10 – Top 10 portas de destino mais acessadas no experimento em ambiente real de média duração. Fonte: Do autor.

Porta Destino	Quantidade de Fluxos	% do Total
443	17291	55.32
53	5691	18.21
80	923	2.95
5228	730	2.34
5222	441	1.41
5353	406	1.30
1900	382	1.22
2408	300	0.96
56552	249	0.80
44296	182	0.58

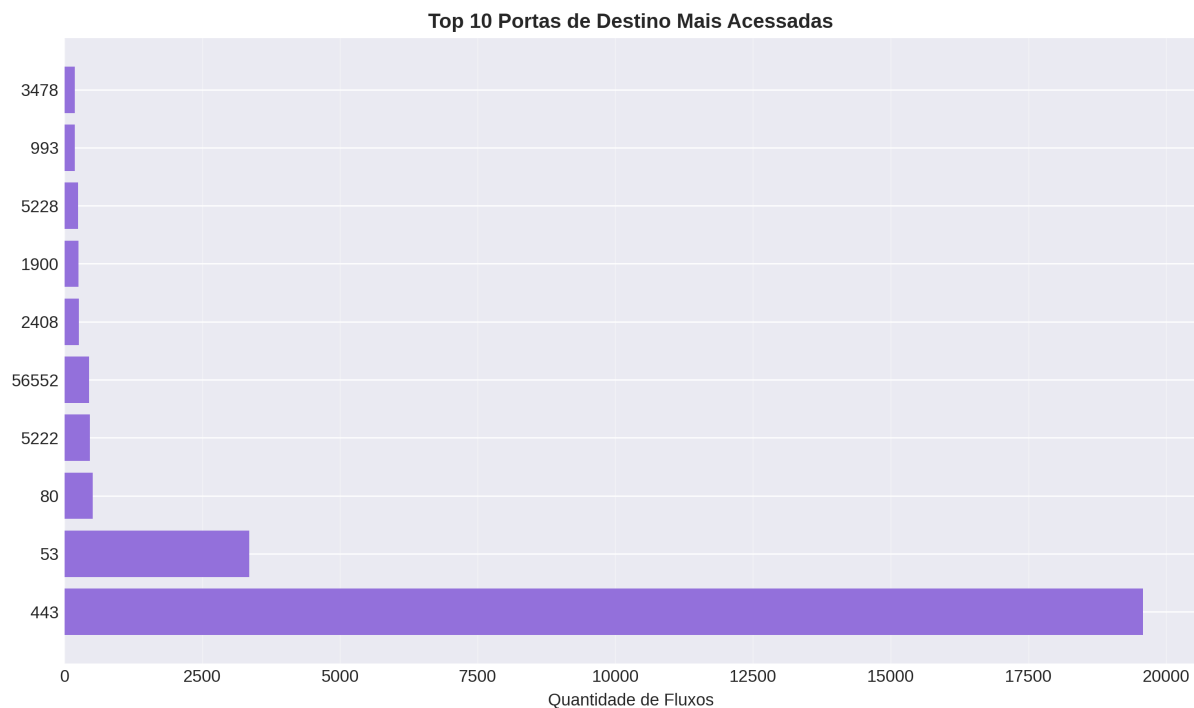


Figura 24 – Visualização das 10 portas de destino com maior volume de tráfego no cenário real de média duração. Fonte: Do autor.

Por fim, a matriz de correlação na Figura 25 atesta a consistência estatística das características extraídas. A forte correlação entre o número de pacotes e o volume de *bytes* trafegados mantém-se inalterada, indicando que a ferramenta *CICFlowmeter* processa corretamente os fluxos, independentemente da origem (real ou simulada) do tráfego. Isso assegura a integridade dos dados para as etapas subsequentes de treinamento de modelos de aprendizado de máquina para detecção de intrusões, ataques ou anomalias.

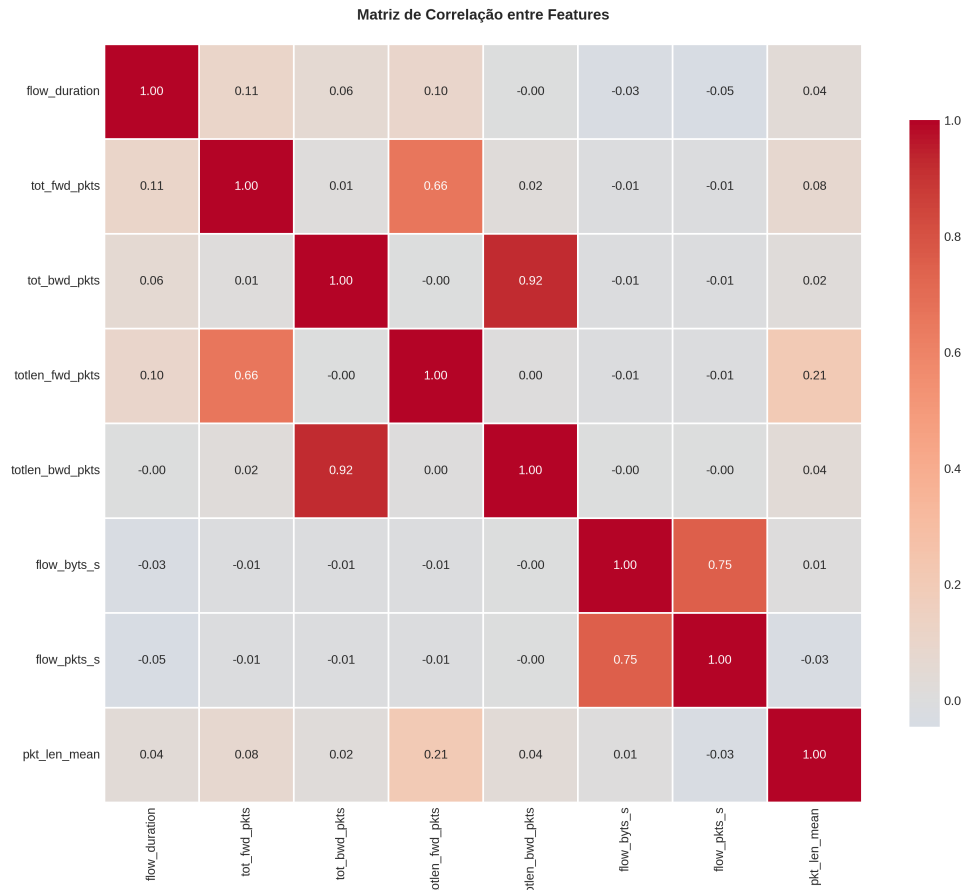


Figura 25 – Matriz de correlação entre as features no experimento de 12 horas em ambiente real. Fonte: Do autor.

A execução deste experimento valida a capacidade da arquitetura de capturar um espectro diversificado de tráfego em um intervalo de tempo intermediário, consolidando a viabilidade do método para a geração de *datasets* híbridos.

4.4.2.2 Experimento de Longa Duração (24 Horas)

O experimento de longa duração em ambiente real, executado ao longo de um ciclo de 24 horas, constitui o teste definitivo de robustez e aplicabilidade da arquitetura proposta. O objetivo central desta etapa é demonstrar como a ferramenta de coleta se comporta diante de um cenário de alta entropia, em que o tráfego gerado por usuários

reais e seus diversos dispositivos domésticos conectados à rede compete por recursos e largura de banda com a infraestrutura de ataques simulados.

A Tabela 11 apresenta os resultados consolidados desta coleta. A análise destes dados revela um aumento expressivo no volume de tráfego, totalizando 53.719 fluxos, o que representa mais que o dobro do volume observado nos experimentos puramente simulados. Este aumento confirma a natureza ruidosa de uma rede doméstica ativa. Além disso, a distribuição de classes indica um cenário de desbalanceamento realista: 76,7% de fluxos normais contra 23,3% de anômalos. Este desequilíbrio é fundamental para treinar sistemas de IDS capazes de operar em ambientes em que o tráfego benigno é vastamente superior ao malicioso.

Tabela 11 – Resumo geral das métricas do experimento em ambiente real de longa duração. Fonte: Do autor.

Métrica	Valor
Duração Total (h)	24.00
Total de Fluxos	53719
Fluxos Normais	41186 (76.7%)
Fluxos Anômalos	12533 (23.3%)
Protocolos Distintos	2
IPs Únicos (Origem)	26
IPs Únicos (Destino)	2055

A representação visual deste desbalanceamento está disponível na Figura 26. O gráfico de setores demonstra claramente a predominância da classe Normal, validando que a inclusão de dispositivos reais (como *smartphones* e computadores pessoais) injeta uma massa de dados legítimos suficiente para diluir, mas não ocultar, os padrões dos *scripts* de automação de ataques.

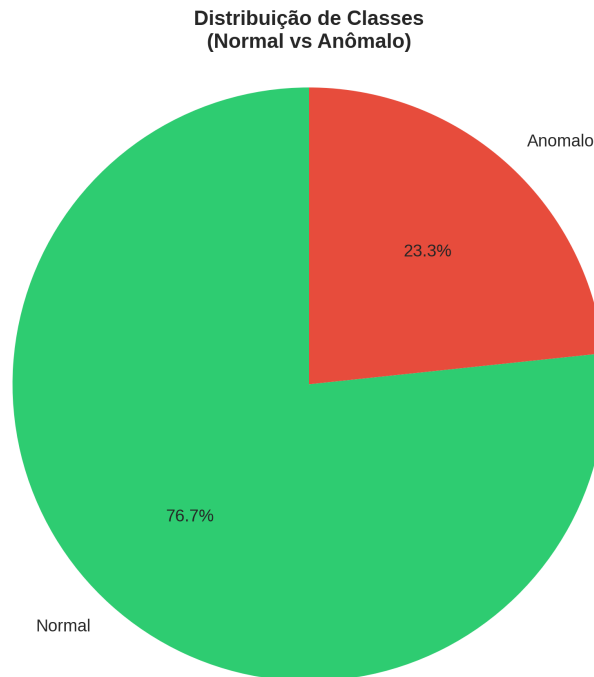


Figura 26 – Distribuição de classes no experimento em ambiente real de longa duração.
Fonte: Do autor.

A dinâmica de uso da rede é evidenciada pela série temporal na Figura 27. Diferentemente dos padrões sintéticos anteriores, este gráfico exhibe picos de tráfego significativamente mais altos, atingindo cerca de 800 fluxos por intervalo de 5 minutos. A variabilidade intensa e a ausência de periodicidade rígida refletem o comportamento humano espontâneo e as comunicações de *background* dos dispositivos conectados. A estabilidade da captura, mesmo diante destes picos abruptos, atesta mais uma vez a resiliência da infraestrutura. Além disso, a Figura 27 evidencia um comportamento rotineiro da rede doméstica, observe que durante a madrugada os fluxos de pacotes tornam-se menos intensos, indicando menor uso da largura de banda. Isso pode ser atrelado quase que diretamente à suspensão do uso destes dispositivos pelos usuários da rede durante este horário, uma vez que é o horário de sono dos residentes.

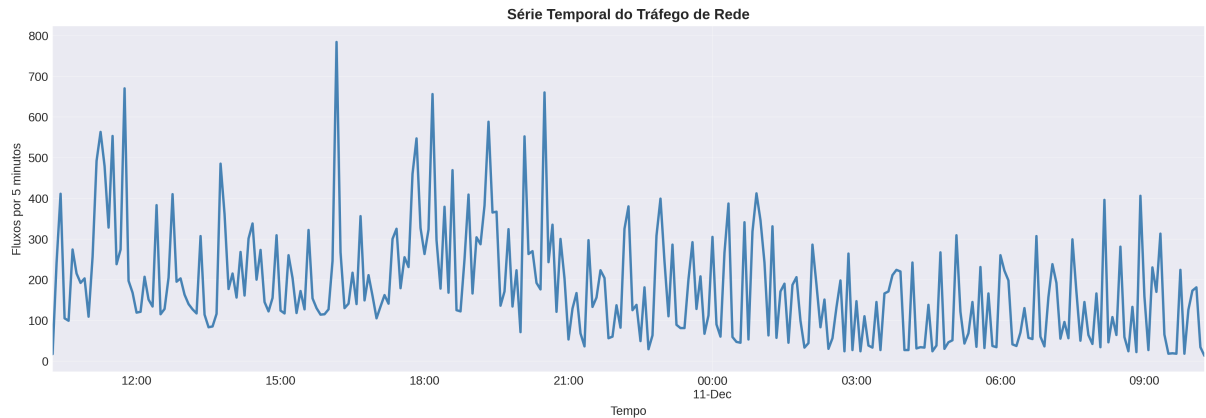


Figura 27 – Série temporal do experimento em ambiente real de longa duração. Fonte: Do autor.

A análise das portas de destino, detalhada na Tabela 12 e na Figura 28, fornece as evidências mais claras da natureza híbrida do ambiente. A porta 443 (HTTPS) domina o tráfego com 57,88% dos fluxos, uma consequência direta do uso de serviços de *streaming*, redes sociais e navegação segura pelos residentes.

Entretanto, o ponto de destaque é a aparição de portas não observadas nos testes artificiais: as portas 5222 (XMPP/*WhatsApp*) e 5228 (*Google Play Store*). A presença destas portas confirma a captura de tráfego proveniente de dispositivos móveis *Android* e aplicativos de mensageria presentes na rede. Adicionalmente, a porta 1900 (SSDP) indica a atividade de descoberta de dispositivos IoT (*Internet of Things*).

Tabela 12 – Top 10 portas de destino mais acessadas no experimento em ambiente real de longa duração. Fonte: Do autor.

Porta Destino	Quantidade de Fluxos	% do Total
443	31094	57.88
53	5278	9.83
80	806	1.50
5222	712	1.33
56552	685	1.28
2408	403	0.75
5228	390	0.73
1900	365	0.68
993	321	0.60
5353	281	0.52

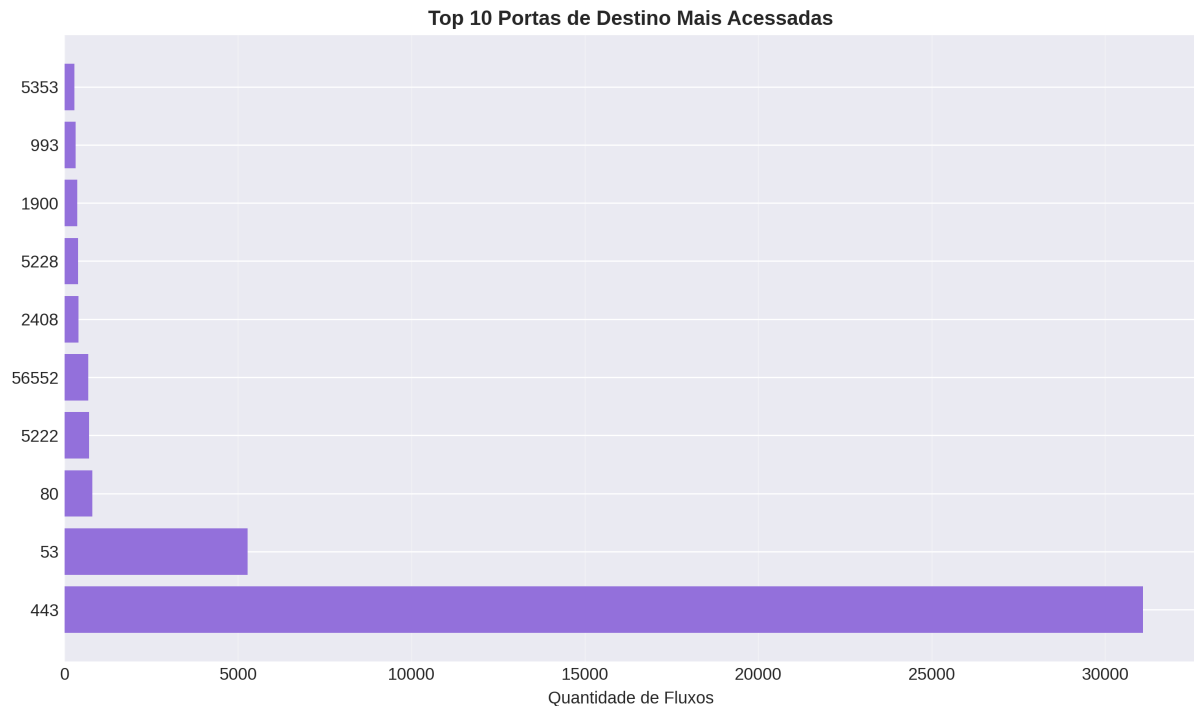


Figura 28 – Visualização das 10 portas de destino com maior volume de tráfego no cenário real. Fonte: Do autor.

Por fim, a matriz de correlação na Figura 29 demonstra que, mesmo com a introdução do ruído e da complexidade do tráfego real, a integridade das *features* extraídas é mantida. As correlações fortes entre contagem de pacotes e volume de bytes persistem, enquanto a independência estatística entre duração do fluxo e taxas de transferência continua válida. Isso garante que o *dataset* final possua a coerência necessária para ser utilizado em algoritmos de aprendizado de máquina, sem sofrer degradação pela heterogeneidade das fontes de tráfego.

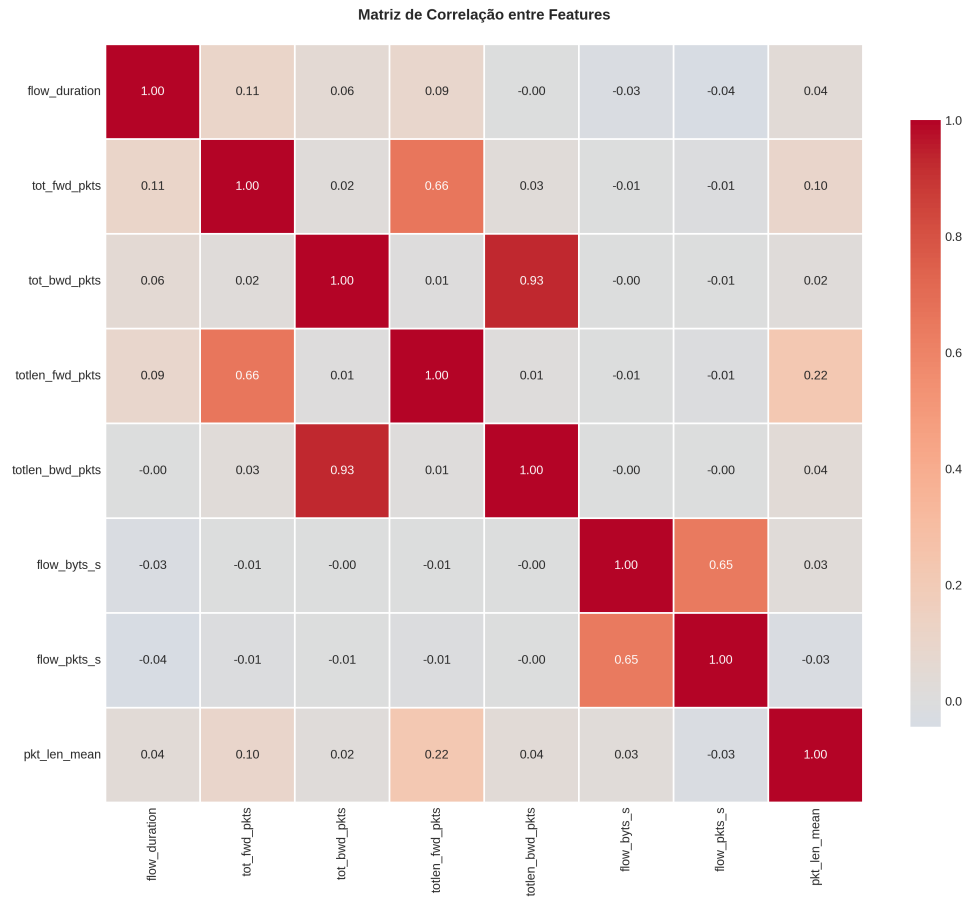


Figura 29 – Matriz de correlação entre as features no experimento de 24 horas em ambiente real. Fonte: Do autor.

A conclusão deste experimento confirma que a arquitetura proposta é capaz de gerar *datasets* híbridos de alta qualidade, capturando a realidade do uso doméstico e enriquecendo-a com dados de ataques controlados, preenchendo assim uma lacuna importante na criação de bases de dados para segurança cibernética.

5 Conclusão

O presente trabalho consolida o desenvolvimento e a validação de uma arquitetura robusta para a geração de *datasets* voltados à segurança cibernética. A infraestrutura proposta, baseada em contêineres *Docker* e segmentação de rede, atinge o objetivo central de criar um ambiente controlado, porém flexível, capaz de simular ataques de rede enquanto captura, simultaneamente, o tráfego legítimo de usuários reais. A eficácia desta abordagem é evidenciada pela qualidade e integridade dos dados obtidos nos experimentos.

A análise dos resultados demonstra que a metodologia de coleta híbrida preenche lacunas importantes na pesquisa de Sistemas de Detecção de Intrusão (IDS), a saber: i) a obsolescência dos *datasets* existentes, que não contemplam padrões de tráfego atuais; ii) a ausência de dados rotulados com *ground truth* confiável; e iii) a carência de conjuntos de dados que integrem tráfego real e ataques controlados. Enquanto os experimentos em ambiente simulado garantem o estabelecimento de um *ground truth* confiável, em que cada fluxo de pacotes é conhecido e rotulado, a introdução do ambiente real confere ao *dataset* a entropia e a complexidade necessárias para simular cenários do mundo real. A diversidade de protocolos observada, com a presença marcante de tráfego móvel e de dispositivos IoT conectados à rede, confirma que a ferramenta captura com fidelidade o comportamento heterogêneo das redes domésticas modernas.

A estabilidade da solução é um dos pontos fortes validados. As séries temporais apresentadas ao longo dos experimentos comprovam a capacidade do *sniffer* em operar de forma contínua por longos períodos, ciclos de 24 horas, sem apresentar lacunas na captura ou degradação de desempenho, mesmo sob picos de tráfego intenso. Este comportamento assegura que o fluxo de dados gerado é contínuo e representa adequadamente a dinâmica temporal da rede, fator crucial para algoritmos de aprendizado de máquina baseados em séries temporais ou janelas de tempo.

Adicionalmente, o balanço de classes obtido no cenário real, com uma predominância natural de tráfego benigno sobre o tráfego anômalo, entrega um cenário de desafio realista para o treinamento de classificadores. Diferentemente de *datasets* sintéticos perfeitamente balanceados, os dados aqui gerados exigem que os modelos de *Machine Learning* lidem com a assimetria típica de ataques reais, aumentando a validade da pesquisa.

As matrizes de correlação, consistentes em todos os cenários, atestam a integridade do processo de extração de características realizado pelo *CICFlowmeter*. A manutenção das relações estatísticas entre as variáveis, independentemente do volume de tráfego ou da origem dos pacotes, indica que o conjunto de dados final possui a coesão necessária para ser utilizado em tarefas de classificação de tráfego.

Portanto, a arquitetura desenvolvida apresenta-se não apenas como uma ferramenta de coleta, mas como um *framework* completo para a criação de *datasets* customizáveis e atualizados. Ela permite que pesquisadores reproduzam cenários de ataque específicos e os misturem com tráfego real, superando a obsolescência natural de bases de dados estáticas antigas e contribuindo diretamente para o avanço das tecnologias de cibersegurança.

5.1 Trabalhos Futuros

A partir dos resultados alcançados e da infraestrutura estabelecida, abrem-se diversas frentes para a continuidade e expansão desta pesquisa. As seguintes propostas visam aprofundar a complexidade dos dados gerados e validar a aplicação prática dos *datasets* em ambientes de produção:

- **Diversificação dos Vetores de Ataque:** A atual implementação foca em ataques de Negação de Serviço (*Slow Loris*) e Mapeamento de Rede. Trabalhos futuros preveem a inclusão de novos *scripts* para simular ataques volumétricos e de exaustão de recursos, como *UDP Floods*, *ICMP Floods* e ataques de força bruta (*Brute Force*) contra serviços SSH e FTP. A escolha destes vetores visa gerar assinaturas de tráfego densas e padrões de fluxo repetitivos, características que favorecem a detecção por algoritmos de aprendizado de máquina baseados em estatísticas de rede.
- **Implementação e Avaliação de Modelos de ML:** O passo natural subsequente é a utilização dos dados gerados para o treinamento efetivo de algoritmos de Aprendizado de Máquina (como *Random Forest*, *SVM* e Redes Neurais Profundas). A avaliação da acurácia, precisão e *recall* destes modelos servirá como a validação definitiva da qualidade do *dataset* produzido.
- **Expansão da Janela Temporal:** Embora o ciclo de 24 horas valide a estabilidade diária, a execução de experimentos de longa duração, na ordem de semanas, permite capturar padrões de comportamento semanais e tendências de longo prazo, úteis para a detecção de anomalias sutis e persistentes.
- **Portabilidade para Nuvem:** A adaptação da arquitetura baseada em *Docker* para orquestradores em nuvem, como *Kubernetes*, possibilita a escalabilidade horizontal dos serviços e a simulação de redes corporativas de grande porte, ampliando o escopo da pesquisa para além das redes domésticas.

Referências

- ARCHIVE, U. K. **KDD Cup 1999 Data**. 1999. Disponível em: <<http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>>. Citado 2 vezes nas páginas 20 e 29.
- BATISTA, G. **Pré-processamento de Dados em Aprendizado de Máquina Supervisionado**. Tese (Doutorado), 01 2003. Citado na página 15.
- COLLEN, A. GHOST - Safe-Guarding Home IoT Environments with Personalised Real-Time Risk Control. In: **Communications in Computer and Information Science**. [S.l.]: Springer Verlag, 2018. p. 68–78. Citado na página 10.
- CYBERSECURITY, C. I. for. **Intrusion Detection Evaluation Dataset (CIC-IDS2017)**. 2017. Accessed: 2025-04-10. Disponível em: <<https://www.unb.ca/cic/datasets/ids-2017.html>>. Citado 4 vezes nas páginas 20, 29, 30 e 34.
- DACIER, M. C.; KONIG, H.; CWALINSKI, R.; KARGL, F.; DIETRICH, S. Security Challenges and Opportunities of Software-Defined Networking. **IEEE Secur. Priv.**, Institute of Electrical and Electronics Engineers (IEEE), v. 15, n. 2, p. 96–100, mar. 2017. Citado na página 10.
- DD-WRT. **DD-WRT - Router Firmware**. 2024. <<https://dd-wrt.com/>>. [Online; accessed 8-June-2024]. Citado 3 vezes nas páginas 11, 17 e 22.
- DuckDNS Org. **Duck DNS - Install**. 2025. Acessado em: 3 aug. 2025. Disponível em: <<https://www.duckdns.org/install.jsp>>. Citado na página 29.
- Evans. Chris. **vsftpd - Secure, fast FTP server for UNIX-like systems**. 2021. Acesso em: 3 aug. 2025. Disponível em: <<https://security.appspot.com/vsftpd.html>>. Citado na página 28.
- FERNANDES, E.; JUNG, J.; PRAKASH, A. Security Analysis of Emerging Smart Home Applications. In: **2016 IEEE Symposium on Security and Privacy (SP)**. [S.l.]: IEEE, 2016. p. 636–654. Citado na página 10.
- KHALID, S.; KHALIL, T.; NASREEN, S. A Survey of Feature Selection and Feature Extraction Techniques in Machine Learning. In: **2014 Science and Information Conference**. [S.l.: s.n.], 2014. Citado na página 19.
- LABS, F. **Fortiguard Threat**. 2022. Acessado em: 2024-05-01. Disponível em: <<https://www.fortinet.com/br/corporate/about-us/newsroom/press-releases/2022/fortiguard-labs-relatorio-ciberataques-brasil-2021>>. Citado na página 10.
- LASHKARI, A. H.; A.KADIR, A. F.; GONZALEZ, H.; MBAH, K. F.; GHORBANI, A. A. Towards a network-based framework for android malware detection and characterization. In: **2017 15th Annual Conference on Privacy, Security and Trust (PST)**. [S.l.: s.n.], 2017. p. 233–23309. Citado na página 30.
- LASHKARI, A. H.; KADIR, A. F. A.; TAHERI, L.; GHORBANI, A. A. Toward developing a systematic approach to generate benchmark android malware datasets

and classification. In: **2018 International Carnahan Conference on Security Technology (ICCST)**. [S.l.: s.n.], 2018. p. 1–7. Citado na página 30.

Lashkari, Arash Habibi. **CICFlowMeter (formerly ISCXFlowMeter)**. 2016. Acessado em: 8 aug. 2025. Disponível em: <<https://www.unb.ca/cic/research/applications.html#CICFlowMeter>>. Citado 2 vezes nas páginas 11 e 30.

_____. **CICFlowMeter (formerly ISCXFlowMeter) Repository**. 2016. Acessado em: 26 aug. 2025. Disponível em: <<https://github.com/ahlashkari/CICFlowMeter>>. Citado na página 33.

Linux Config Org. **Setting a Static Ip Address in Ubuntu 20.04 via the Command Line**. 2024. Acessado em: 3 aug. 2025. Disponível em: <<https://linuxconfig.org/setting-a-static-ip-address-in-ubuntu-24-04-via-the-command-line>>. Citado na página 28.

MARAYA, M. **port-mirroring: OpenWrt package for copying network packets without iptables**. [S.l.]: GitHub, 2024. <<https://github.com/mmaraya/port-mirroring>>. Accessed: 2025-06-10. Citado 2 vezes nas páginas 23 e 26.

MARIAN, T. E.; DORU, B. Improving Network Security Using DD-WRT as a Solution for SOHO Routers. In: **2023 22nd RoEduNet Conference: Networking in Education and Research (RoEduNet)**. [S.l.: s.n.], 2023. p. 1–5. Citado na página 17.

MASCARENHAS, C.; PRASAD, R.; BORGES, P.; SYED, S. F. Project Urban Patrol: Building an Attack Resilient Smart Home Architecture. In: **2021 4th Biennial International Conference on Nascent Technologies in Engineering (ICNTE)**. [S.l.]: IEEE, 2021. Citado na página 10.

MAĆKIEWICZ, A.; RATAJCZAK, W. Principal components analysis (PCA). **Computers Geosciences**, v. 19, n. 3, p. 303–342, 1993. ISSN 0098-3004. Disponível em: <<https://www.sciencedirect.com/science/article/pii/009830049390090R>>. Citado na página 16.

MEHR, I. E.; MINETTO, A.; DOVIS, F.; PICA, E.; CESARONI, C.; ROMANO, V. **An Open Architecture for Signal Monitoring and Recording Based on SDR and Docker Containers: A GNSS Use Case**. 2023. Citado na página 36.

MOUSTAFA, N.; SLAY, J. The Significant Features of the UNSW-NB15 and the KDD99 Data Sets for Network Intrusion Detection Systems. In: **2015 4th International Workshop on Building Analysis Datasets and Gathering Experience Returns for Security (BADGERS)**. [S.l.: s.n.], 2015. p. 25–31. Citado na página 20.

MOUSTAFA, N.; SLAY, J. **UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems**. 2015. Accessed: 2025-04-10. Disponível em: <<https://research.unsw.edu.au/projects/unsw-nb15-dataset>>. Citado 2 vezes nas páginas 20 e 29.

NESTAR. 1984. <<https://archive.org/details/nestar-arcnet-sniffer-internal-descriptions/mode/2up>>. Citado na página 15.

- Nguyen. Dat-Thinh. **CICFlowMeter Python Repository**. 2022. Acessado em: 26 aug. 2025. Disponível em: <<https://github.com/datthinh1801/cicflowmeter>>. Citado na página 33.
- Nmap Org. **Nmap: the Network Mapper - Free Security Scanner**. 2025. Acessado em: 3 aug. 2025. Disponível em: <<https://nmap.org/>>. Citado 4 vezes nas páginas 29, 42, 43 e 49.
- OPENARGUS. **OpenARGUS: Open Source Home Network Supervision**. 2024. Accessed: 2025-04-10. Disponível em: <<https://openargus.org/>>. Citado 2 vezes nas páginas 19 e 30.
- OpenWrt Project... **OpenWrt - Archer C6 v3 - OEM easy installation**. 2024. Acesso em: 31 april. 2025. Disponível em: <https://openwrt.org/toh/tp-link/archer_c6_v3#oem_easy_installation>. Citado na página 23.
- OpenWrt Project.... **OpenWrt - Enabling a Wi-Fi access point on OpenWrt**. 2024. Acesso em: 31 april. 2025. Disponível em: <https://openwrt.org/docs/guide-quick-start/basic_wifi#setup_with_the_web_gui>. Citado na página 27.
- _____. **OpenWrt - Opkg Package Manager**. 2024. Acesso em: 22 july. 2025. Disponível em: <<https://openwrt.org/docs/guide-user/additional-software/opkg>>. Citado na página 26.
- OpenWrt Project. **OpenWrt - Wireless Freedom**. 2024. <<https://openwrt.org/>>. [Online; accessed 8-June-2024]. Citado 4 vezes nas páginas 11, 14, 17 e 22.
- OpenWrt Project.. **OpenWrt Table of Hardware - Device Compatibility Guide**. 2025. <<https://toh.openwrt.org/?view=normal>>. Accessed: 2025-06-10. Citado na página 23.
- OpenWrt Project. **Supported Devices**. 2025. <https://openwrt.org/supported_devices>. Accessed: 2025-06-10. Citado na página 23.
- Raspberry Pi. **Raspberry Pi - Getting Started**. 2025. Acesso em: 3 aug. 2025. Disponível em: <<https://www.raspberrypi.com/documentation/computers/getting-started.html>>. Citado na página 28.
- Raspberry Pi. **Raspberry Pi - Model 4B**. 2025. Acesso em: 3 aug. 2025. Disponível em: <<https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>>. Citado na página 28.
- SHARMA, P. K.; PARK, J. H.; JEONG, Y.-S.; PARK, J. H. SHSec: SDN based secure smart home network architecture for internet of things. **Mob. Netw. Appl.**, Springer Science and Business Media LLC, v. 24, n. 3, p. 913–924, jun. 2019. Citado na página 10.
- SIKOS, L. F. Packet Analysis for Network Forensics: A Comprehensive Survey. **Forensic Science International: Digital Investigation**, v. 32, p. 200892, 2020. ISSN 2666-2817. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S1742287619302002>>. Citado na página 18.
- The Tcpdump Group. **The Tcpdump Group Website**. 2025. <<https://www.tcpdump.org/index.html>>. Acessado em: 10 abr. 2025. Citado 2 vezes nas páginas 18 e 19.

- Tshark. **TShark.dev — TShark Documentation and Usage**. 2024. Acesso em: 1 jul. 2025. Disponível em: <<https://tshark.dev/>>. Citado 4 vezes nas páginas 11, 14, 15 e 29.
- Ubuntu Community. **vsftpd - Community Help Wiki**. 2015. Acessado em: 3 aug. 2025. Disponível em: <<https://help.ubuntu.com/community/vsftpd>>. Citado na página 28.
- Ubuntu Community. **UFW - Community Help Wiki**. 2023. Acesso em: 3 aug. 2025. Disponível em: <<https://help.ubuntu.com/community/UFW>>. Citado na página 29.
- Ubuntu Releases. **Ubuntu 20.04.6 LTS (Focal Fossa)**. 2025. Acesso em: 3 aug. 2025. Disponível em: <<https://releases.ubuntu.com/focal/>>. Citado na página 28.
- VENTINO, G. **ProjectBeholder: Domestic IDS Dataset Tools**. [S.l.]: GitHub, 2026. <<https://github.com/gventino/ProjectBeholder>>. Accessed: 2026-01-17. Citado na página 22.
- WIRESHARK. **Wireshark · Go Deep**. 2024. <<https://www.wireshark.org/>>. [Online; acessado em 4 de junho de 2024]. Citado 3 vezes nas páginas 15, 18 e 19.
- YALTIRAKLI, G. Slowloris. **github.com**, 2015. Disponível em: <<https://github.com/gkbrk/slowloris>>. Citado 3 vezes nas páginas 45, 48 e 49.
- ZAIDAN, A. A.; ZAIDAN, B. B. A Review on Intelligent Process for Smart Home Applications Based on IoT: Coherent Taxonomy, Motivation, Open Challenges, and Recommendations. **Artif. Intell. Rev.**, Springer Science and Business Media LLC, jul. 2018. Citado na página 10.